



**COORDENADORIA DE ENGENHARIA DA
COMPUTAÇÃO**

GUSTAVO DE SOUZA MARQUES

**MIDDLEWARE PARA BALANCEAMENTO DE CARGA INTELIGENTE
EM APIS REST**

**SOROCABA, SP
2023**



GUSTAVO DE SOUZA MARQUES

**MIDDLEWARE PARA BALANCEAMENTO DE CARGA INTELIGENTE
EM APIS REST**

Trabalho de conclusão de curso
apresentado ao Centro Universitário
Facens como exigência parcial para
obtenção do diploma de graduação em
Engenharia da Computação.

Orientador (a): Deivison Shindi Takatu.

**SOROCABA, SP
2025**

FICHA CATALOGRÁFICA ELABORADA PELA
“BIBLIOTECA FACENS”

Bibliotecária responsável
Eliane da Rocha CRB 8062/8a

M357m

Marques, Gustavo de Souza.

Middleware para balanceamento de carga em API's REST / por
Gustavo de Souza Marques. - Sorocaba, SP: [s.n.], 2025.
58f.

Trabalho de Conclusão de Curso (Graduação) – Centro
Universitário Facens – Coordenadoria de Engenharia da
Computação - Curso de Engenharia da Computação, 2025.
Orientador: Prof. Me. Deivison Shindi Takatu.

1. Middleware. 2. API REST. 3. Balanceamento de Carga. I.
Centro Universitário Facens. IV. Título.

CDD 621.39

GUSTAVO DE SOUZA MARQUES

**MIDDLEWARE PARA BALANCEAMENTO DE CARGA INTELIGENTE
EM APIS REST**

Trabalho de conclusão de curso apresentado
ao Centro Universitário Facens como
exigência parcial para obtenção do certificado
de conclusão do curso de Engenharia da
Computação.

Orientador : Deivison Shindi Takatu.

BANCA EXAMINADORA

Prof. Me. Deivison Shindi Takatu

Prof. Esp. Marco Antonio Montebello Junior

Prof. Me. Rafael Soleiman Franco

AGRADECIMENTOS

Agradeço a esta instituição de ensino por me preparar para o mercado de trabalho, a todos os professores, coordenadores, colegas de classe e funcionários em geral que de alguma forma estiveram comigo nesta trajetória.

Aos meus familiares, que depositaram muita confiança e credibilidade em mim e em meus planos.

Aos orientadores pelo incentivo disponibilizado, ao tempo doado para que eu pudesse encontrar a melhor forma de concluir o meu projeto.

A todos integrantes da biblioteca da faculdade pela atenção e carinhoso suporte.

Meus sinceros agradecimentos.

RESUMO

Neste trabalho, apresentamos a criação e a avaliação de um sistema simples que tenta distribuir melhor as requisições entre várias instâncias de uma API. O objetivo é claro: impedir que um servidor fique sobrecarregado enquanto outros permanecem ociosos, situação que costuma gerar respostas lentas e erros nos momentos de pico. Para alcançar isso, desenvolvemos um middleware em .NET e testamos quatro maneiras de encaminhar as requisições: Round-Robin, Least-Connections, Load-Based e um modelo híbrido adaptativo. Montamos cenários que imitam situações cotidianas – uso constante, picos repentinos, degradação parcial de uma instância e diminuição gradual do tráfego. Em cada um deles analisamos três aspectos essenciais: tempo de resposta, quantidade de requisições atendidas e taxa de erros. Também registramos, para cada resposta, qual instância efetivamente realizou o atendimento, a fim de confirmar se a distribuição realmente ficou equilibrada. Os números vieram com uma coerência surpreendente: as técnicas que se calibram ao estado do sistema — particularmente a baseada em carga e a versão híbrida — conseguiram respostas mais velozes e apresentaram menos falhas tanto ao enfrentar picos de demanda quanto ao sofrer interrupções parciais, sem sacrificar a taxa de atendimento. O método Least-Connections mostrou-se eficaz em situações de desbalanceamento sutil, enquanto o Round-Robin, embora tenha proporcionado um impulso inicial graças à sua simplicidade, não se ajustou às disparidades entre as instâncias. O estudo foi arquitetado para ser reproduzível: todo o cenário de testes e os scripts de análise foram orquestrados de forma que outras equipes possam replicar e comparar os achados. Como próximos passos, propomos ampliar os testes a outros endpoints, diversificar os tamanhos das requisições e simular condições de rede que se assemelhem mais ao ambiente de produção.

Palavras-chave: balanceamento de carga; middleware; .NET; API REST; resiliência.

ABSTRACT

In this work, we present the creation and evaluation of a simple system that attempts to better distribute requests among multiple instances of an API. The objective is clear: to prevent one server from becoming overloaded while others remain idle, a situation that usually generates slow responses and errors during peak times. To achieve this, we developed middleware in .NET and tested four ways to route requests: Round-Robin, Least-Connections, Load-Based, and an adaptive hybrid model. We created scenarios that mimic everyday situations – constant use, sudden peaks, partial degradation of an instance, and gradual decrease in traffic. In each scenario, we analyzed three essential aspects: response time, number of requests handled, and error rate. We also recorded, for each response, which instance actually handled the request, in order to confirm whether the distribution was truly balanced. The numbers came with surprising consistency: techniques that calibrate to the system state—particularly the load-based and hybrid versions—achieved faster responses and presented fewer failures both when facing peak demand and when experiencing partial interruptions, without sacrificing the service rate. The Least-Connections method proved effective in situations of subtle imbalance, while Round-Robin, although it provided an initial boost thanks to its simplicity, did not adjust to the disparities between instances. The study was designed to be reproducible: the entire test scenario and analysis scripts were orchestrated so that other teams can replicate and compare the findings. As next steps, we propose expanding the tests to other endpoints, diversifying the request sizes, and simulating network conditions that more closely resemble the production environment.

Keywords: Load Balancing; Middleware; .NET; REST API; resilience.

LISTA DE ILUSTRAÇÕES

Figura 1 - Exemplo de Balanceamento Round Robin. (Fonte: Disponível em: https://somasagility.com.br . Acesso em: 06 mai. 2025.)	26
Figura 2 - Balanceamento por Menor Número de Conexões (Least Connections)	27
Figura 3 – Fluxo Metodológico da Pesquisa	32
Figura 4 – Arquitetura Lógica do Middleware em ASP.NET Core	35
Figura 5 - Latência p95 (ms) por estratégia no cenário de carga estável (C1) ..	46
Figura 6 - Throughput (req/s) por estratégia no cenário de carga estável (C1) ..	47
Figura 7 - Distribuição de carga por backend no cenário de carga estável (C1) ..	48
Figura 8 - Latência p95 (ms) por estratégia no cenário de pico abrupto de carga (C2)	49
Figura 9 - Throughput (req/s) por estratégia no cenário de pico abrupto de carga (C2)	50
Figura 10 - Distribuição de carga por backend no cenário de pico abrupto de carga (C2)	51
Figura 11 - Latência p95 (ms) por estratégia no cenário de degradação parcial de instância (C3)	52
Figura 12 - Throughput (req/s) por estratégia no cenário de degradação parcial de instância (C3)	53
Figura 13 - Distribuição de carga por backend no cenário de degradação parcial de instância (C3)	54
Figura 14 - Latência p95 (ms) por estratégia no cenário de ramp-down (C4) ..	55
Figura 15 - Throughput (req/s) por estratégia no cenário de ramp-down (C4) ..	56
Figura 16 - Distribuição de carga por backend no cenário de ramp-down (C4) ..	57
Figura 17 - Latência p95 (ms) média global por estratégia considerando todos os cenários.....	58
Figura 18 - Throughput (req/s) médio global por estratégia considerando todos os cenários.....	59

LISTA DE SIGLAS

API	<i>Application Programming Interface</i> — Interface de Programação de Aplicações
ASP.NET Core	<i>Active Server Pages .NET Core</i> — Framework multiplataforma para desenvolvimento de aplicações web em .NET
AWS	<i>Amazon Web Services</i> — Plataforma de computação em nuvem da Amazon
CPU	<i>Central Processing Unit</i> — Unidade Central de Processamento
CRUD	<i>Create, Read, Update, Delete</i> — Operações básicas em bancos de dados e APIs
DNS	<i>Domain Name System</i> — Sistema de Nomes de Domínio
GET	Método HTTP utilizado para obter informações de um recurso
HTTP	<i>Hypertext Transfer Protocol</i> — Protocolo de comunicação para transferência de dados na web
HTTPS	<i>Hypertext Transfer Protocol Secure</i> — Versão segura do protocolo HTTP
JSON	<i>JavaScript Object Notation</i> — Formato leve de intercâmbio de dados
LB	<i>Load Balancing</i> — Balanceamento de carga
REST	<i>Representational State Transfer</i> — Estilo de arquitetura de software baseado em recursos
SLA	<i>Service Level Agreement</i> — Acordo de Nível de Serviço
SQL	<i>Structured Query Language</i> — Linguagem estruturada para gerenciamento de bancos de dados
URI	<i>Uniform Resource Identifier</i> — Identificador uniforme de recurso
URL	<i>Uniform Resource Locator</i> — Endereço de recurso na web
XML	<i>Extensible Markup Language</i> — Linguagem de marcação extensível usada para estruturar dados

LISTA DE SÍMBOLOS

L_avg	Latência média observada nas requisições (ms)
L_p95	Latência p95 das requisições (ms)
L_p99	Latência p99 das requisições (ms)
T	Throughput, número de requisições processadas por segundo
N	Número de instâncias de backend ativas
req/s	Requisições por segundo
ms	Milissegundo

SUMÁRIO

COORDENADORIA DE ENGENHARIA DA COMPUTAÇÃO	1
1 INTRODUÇÃO	17
2 BALANCEAMENTO DE CARGA: CONCEITOS E CONCEPÇÕES	21
2.1 ARQUITETURA REST E APIS WEB	22
2.2 CONCEITO DE MIDDLEWARE	22
2.3 BALANCEAMENTO DE CARGA EM SISTEMAS DISTRIBUÍDOS	23
2.4 ESTRATÉGIAS DE BALANCEAMENTO DE CARGA	23
2.4.1 Algoritmo Round Robin	25
2.4.2 Algoritmo Least Connections	26
2.4.3 Algoritmo Load-Based	27
2.4.4 Algoritmo Híbrido	28
2.5 Middleware Inteligente para Balanceamento de Carga em APIs REST	30
3 METODOLOGIA	31
3.1 Tipo de Pesquisa	31
3.2 Abordagem Metodológica	31
3.3 Etapas da Pesquisa	31
3.4 Ferramentas e Tecnologias Utilizadas	33
3.5 Arquitetura do Middleware	33
3.6 Métricas de Avaliação	36
3.7 Cenários de Validação	37
3.8 Limitações do Estudo	42
4 RESULTADOS	43
4.1 Visão geral dos experimentos	43
4.2 Cenário C1 – Carga estável	46
4.3 Cenário C2 – Pico abrupto de carga (flash crowd)	48
4.4 Cenário C3 – Degradação parcial de instância	51
4.5 Cenário C4 – Ramp-down de usuários	54
4.6 Comparação global entre estratégias	58
5 CONSIDERAÇÕES FINAIS	61
REFERÊNCIAS	64

1 INTRODUÇÃO

Atualmente as aplicações distribuídas têm se tornado cada vez mais predominantes na criação de sistemas atualizados e avançados devido aos avanços na computação em nuvem, nas práticas DevOps e de microsserviços. Dentro desse contexto dinâmico de evolução tecnológica e colaboração entre os componentes do sistema online em tempo real com eficiência otimizada pelas APIs RESTful que desempenham um papel crucial ao facilitar a comunicação de maneira simples e padronizada entre os diferentes elementos do sistema utilizando tecnologias leves como HTTPS e JSON. Uma maneira comum de compreender uma API REST é vê-la como uma ligação entre o cliente e um conjunto de recursos em que cada solicitação é autônoma e não requer a manutenção do estado no servidor.

Sua relevância é tão significativa que atualmente muitas aplicações web e móveis dependem das APIs REST para suas funcionalidades essenciais. Porém com o aumento exponencial do número de requisições e usuários simultâneos, a garantia do desempenho dessas APIs tornou-se um desafio continuamente presente.

Muitas vezes esse desafio se torna mais complexo quando os servidores de *back-end* precisam lidar com picos de tráfego ou variações intensas na carga de trabalho. Mesmo com a replicação das instâncias de *back-end*, é fundamental ter um mecanismo inteligente capaz de distribuir essas solicitações entre os servidores disponíveis para evitar subutilização em alguns e sobrecarga em outros servidores. Esse mecanismo é conhecido como balanceamento de carga e sua implementação correta impactar diretamente na experiência do usuário final e na eficiência da infraestrutura usada pela aplicação.

De maneira geralmente observada em várias situações práticas cotidianas atuais de uso de tecnologia da informação e comunicação (TIC), muitas aplicações se utilizam comumente de sistemas ou mecanismos de balanceamento de carga convencionais e tradicionais para distribuir equitativamente as demandas ou solicitações entre diferentes servidores ou recursos computacionais disponíveis na

rede. Para exemplificar essa questão com um enfoque técnico do ponto de vista da engenharia da computação e ciência da computação; por exemplo o balanceador *Round Robin* é uma técnica comum na programação utilizada para distribuir cargas sequenciais em servidores em sistemas operacionais colaborativos e/ou conectados em rede local. Entretanto é importante ressaltar que mesmo sendo amplamente utilizadas, essas técnicas tradicionais apresentam certas limitações principalmente em ambientes computacionais complexos onde os servidores possuem disparidades significativas no desempenho individual. Ademais, é válido mencionar que abordagens baseadas unicamente no número absoluto de conexões ativas como ocorre no algoritmo *Least Connections* também podem não ser totalmente eficientes pois não levam em consideração aspectos críticos como o uso efetivo do processador (CPU) ou latência real percebida pela infraestrutura física da rede telemática. Agora há uma demanda por solucionar de forma flexível e tomar decisões com base em métricas em tempo real.

O desafio deste projeto reside na procura por uma distribuição de carga eficiente para as APIs REST que leve em consideração não apenas o volume de requisições ou conexões recebidas, mas também a capacidade real de resposta de cada servidor *backend* envolvido. Essencialmente importante em sistemas operando sob variações de carga como plataformas de comércio eletrônico ou serviços de streaming ou ainda aplicações governamentais onde falhas no desempenho podem acarretar prejuízos.

Em muitas situações práticas não é suficiente seguir a maneira tradicional de equilibrar solicitações; Especialmente quando se busca flexibilidade operacional eficiência contínua dos serviços de TI de forma constante é necessário adotar novas estratégias. Por isso, neste estudo propõem-se o desenvolvimento de um middleware inteligente escrito em C#, que é capaz de gerenciar a distribuição das cargas entre os servidores REST utilizando vários algoritmos diferentes incluindo uma nova metodologia baseada no desempenho (*Load-Based*). Com isso, busca-se analisar como cada abordagem influencia o tempo de resposta, os índices de sucesso e a utilização de recursos computacionais.

O motivo da escolha desse tema é sua importância prática atual no campo da engenharia de *software* e da arquitetura de sistemas escaláveis. Com o aumento do uso de APIs como peça fundamental na comunicação entre serviços, tornou-se crucial estudar alternativas que assegurem sua estabilidade. Estudos recentes mostram que APIs são hoje o principal mecanismo de troca de dados e de conexão entre sistemas, inclusive em arquiteturas baseadas em *microserviços*, desempenhando papel central na integração de aplicações e na economia digital (MOHITE, 2024). Além disso, esse assunto possui um grande valor acadêmico, pois possibilita aplicar conceitos teóricos de sistemas distribuídos, análise de desempenho desenvolvimento web com C#. O interesse pelo assunto devido à demandas de sistemas empresariais em projetos com problemas de requisições HTTP, nos quais sistemas automatizados de balanceamento poderiam ter evitado falhas de disponibilidade.

Para definir o escopo deste trabalho, definiu-se como problema central a ser investigado a seguinte questão: “Em um ambiente de APIs REST com múltiplas instâncias de *backend*, submetidas a diferentes perfis de carga e eventuais degradações parciais, um middleware de balanceamento de carga híbrido adaptativo é capaz de oferecer desempenho e estabilidade superiores a algoritmos tradicionais amplamente utilizados na prática?”

A partir dessa formulação, estabelece-se a seguinte hipótese de pesquisa:

Um middleware de balanceamento de carga híbrido adaptativo, que combina heurísticas *Round-Robin*, *Least-Connections* e *Load-Based* guiadas por métricas de monitoramento em tempo quase real, tende a reduzir a latência de cauda (p95/p99) e a variabilidade dos resultados entre execuções, além de distribuir melhor o tráfego em cenários de degradação parcial de instâncias, quando comparado a um cenário baseline sem balanceamento explícito.

O objetivo geral do trabalho foi projetar, implementar e avaliar experimentalmente um middleware em .NET capaz de aplicar, de forma comparativa, as quatro estratégias de roteamento (*Baseline* sem balanceamento, *Round-Robin*, *Least-Connections*, *Load-Based* e Híbrida Adaptativa) em um ambiente controlado de APIs REST. Os objetivos específicos incluem:

- modelar uma arquitetura de testes baseada em contêineres Docker, contemplando cliente gerador de carga, *middleware*, instâncias *backend* de e componentes de observabilidade;
- definir um conjunto de métricas quantitativas alinhadas a acordos de nível de serviço (SLA), incluindo latência p95/p99, *throughput*, taxa de erros e distribuição de requisições por instância;
- elaborar cenários de carga que representem situações típicas de produção (carga estável, pico abrupto, degradação de instância e *ramp-down*);
- analisar comparativamente os resultados obtidos, discutindo em quais condições a estratégia híbrida adaptativa apresenta vantagens significativas sobre o baseline e sobre as demais políticas estáticas/dinâmicas avaliadas.

Para embasar teoricamente o estudo foram escolhidos pesquisadores renomados e trabalhos recentes que discutem tanto os princípios dos sistemas distribuídos quanto as aplicações práticas do balanceamento de carga em arquiteturas REST. Entre os autores proeminentes consultados estão Costa et al (2023), Ahmed et al (2022), Al-Badameh et al (2021), Zhou et al (2022). Hassan e colaboradores (2022) exploraram algoritmos adaptativos e métricas de desempenho em ambientes nativos de nuvem.

2 BALANCEAMENTO DE CARGA: CONCEITOS E CONCEPÇÕES

O balanceamento de carga é um dos elementos centrais para garantir o desempenho e a escalabilidade de sistemas distribuídos contemporâneos. À medida que aplicações web e serviços digitais se tornam mais complexos e dependentes de múltiplos servidores, surge a necessidade de distribuir dinamicamente as requisições de modo a evitar sobrecargas e falhas em componentes individuais. Segundo Al-Badarnah, Harfoushi e Al-Jarrah (2021), a evolução das tecnologias de nuvem e o aumento exponencial das transações online tornaram o balanceamento de carga uma prática indispensável para assegurar estabilidade e desempenho em ambientes distribuídos.

De forma geral, o balanceamento de carga consiste na divisão inteligente do tráfego de rede entre vários servidores ou instâncias de *backend*, com o objetivo de maximizar a eficiência do sistema e reduzir o tempo de resposta percebido pelo usuário. Ahmed, Riaz e Ali (2022) destacam que a escolha do algoritmo de distribuição influencia diretamente na capacidade do sistema de responder a variações na demanda, impactando a disponibilidade e a qualidade do serviço (QoS).

No contexto das APIs RESTful, o desafio de manter o equilíbrio entre desempenho, escalabilidade e confiabilidade é ainda mais evidente. Isso ocorre porque esse tipo de arquitetura, amplamente adotado em aplicações modernas, é responsável por intermediar praticamente todas as comunicações entre serviços e sistemas heterogêneos. Conforme enfatizado por Zhou, Jiang e Wang (2022), a introdução de *middlewares* inteligentes, capazes de adaptar sua estratégia de balanceamento conforme as condições de execução, representa um passo essencial para o avanço de sistemas distribuídos autogerenciáveis e resilientes.

Portanto, compreender os fundamentos do balanceamento de carga e suas concepções técnicas é fundamental para embasar o desenvolvimento de soluções inovadoras, como o *middleware* proposto neste trabalho. A seguir, será apresentada uma visão detalhada da arquitetura REST e do papel das APIs Web, que constituem

a base de comunicação sobre a qual as estratégias de balanceamento serão aplicadas.

2.1 ARQUITETURA REST E APIS WEB

Segundo Rodrigues et al.(2021), o modelo de arquitetura REST (Representational State Transfer) é amplamente adotado para construir sistemas distribuídos que utilizam APIs na comunicação entre aplicações web devido à sua ênfase em interações sem estado e no gerenciamento de recursos por meio de operações HTTP para garantir escalabilidade e interoperabilidade.

O crescimento das APIs públicas e privadas tem sido impulsionado pela disseminação das arquiteturas RESTful, conforme apontado por Richardson e Amundsen (2018), demandando a adoção de práticas atualizadas para lidar com escalabilidade e alta disponibilidade. Com o aumento significativo no número de requisições simultâneas, tornou-se vital implementar estratégias eficientes de balanceamento de carga para garantir a qualidade do serviço (QoS).

2.2 CONCEITO DE MIDDLEWARE

Middleware pode ser descrito como um tipo de software que suporta a troca de informações e no controle de dados entre diferentes aplicações de forma simplificada e sem lidar diretamente com a complexidade da rede (da Silva; Barbosa 2020). Em sistemas RESTful APIs (APIs Representacionais de Estado Transferido), o middleware assume funções importantes como agir como um intermediário inteligente ao oferecer recursos como direcionamento dinâmico do tráfego web autenticação segura e equilíbrio eficiente da carga de trabalho.

De acordo com o estudo de Hassan et al. (2022), é crucial que os middlewares atuais sejam desenvolvidos considerando a flexibilidade e capacidade de se adaptar e ser observado - aspectos essenciais para ambientes nativos em nuvem (*cloud-native*) e arquiteturas baseadas em microsserviços.

2.3 BALANCEAMENTO DE CARGA EM SISTEMAS DISTRIBUÍDOS

A distribuição de carga é essencial para garantir que sistemas distribuídos possam dimensionar horizontalmente de forma eficiente. Se concentra em distribuir de maneira equitativa o tráfego entre as instâncias disponíveis para evitar gargalos e sobrecargas desnecessárias (Kumar; Tripathi - 2020).

Agora as estratégias de equilíbrio têm progredido para integrar inteligência contextual fundamentada em indicadores de desempenho e utilização de recursos tornando-se ajustáveis em tempo real (Al-Badarneh ; Harfoushi ; Al-Jarrah 2021).

2.4 ESTRATÉGIAS DE BALANCEAMENTO DE CARGA

A seleção da estratégia de balanceamento é decisiva para desempenho e escalabilidade em sistemas distribuídos. Em arquiteturas baseadas em APIs REST, onde múltiplas instâncias atendem requisições concorrentes, o balanceador atua como elo intermediário responsável por distribuir tráfego de forma justa, estável e responsiva, evitando gargalos locais e subutilização de recursos (AL-BADARNEH; HARFOUSHI; AL-JARRAH, 2021). Em linhas gerais, distinguem-se abordagens estáticas — com regras fixas e baixo custo operacional — e dinâmicas, que ajustam o roteamento a partir de métricas em tempo real (número de conexões ativas, latência, uso de CPU), o que as torna mais adequadas a ambientes variáveis e heterogêneos (AHMED; RIAZ; ALI, 2022).

A literatura recente aponta que políticas estritamente fixas podem ser insuficientes quando há assimetria entre instâncias (capacidade de *hardware*, localização, estado de rede) ou variações súbitas de demanda; nessas condições, aumentam as filas locais e a cauda de latência, comprometendo a QoS (ZHOU; JIANG; WANG, 2022). Em resposta, surgiram esquemas adaptativos e híbridos que combinam múltiplos sinais de carga. Há, inclusive, propostas que integram aprendizado de máquina e telemetria contínua para antecipar picos e ajustar pesos de forma proativa; embora promissoras, essas abordagens trazem complexidade de integração e dependência de dados de alta qualidade, demandando avaliação cuidadosa antes do uso em produção (COSTA; MOREIRA; VASCONCELOS, 2023).

Justificativa das estratégias analisadas. À luz desse panorama, este trabalho compara três estratégias amplamente aceitas na prática e na literatura — *Round-Robin* (RR), *Least-Connections* (LC) e *Load-Based* (LB) — por três razões principais:

Cobertura conceitual do espectro estático-dinâmico.

- RR representa o polo estático: simples, determinístico e com custo mínimo, apropriado como baseline em cenários relativamente homogêneos (AL-BADARNEH; HARFOUSHI; AL-JARRAH, 2021).
 - LC introduz consciência de estado ao considerar conexões ativas, mitigando acúmulos transitórios e desbalanceamentos leves (AHMED; RIAZ; ALI, 2022).
 - LB incorpora sinais de desempenho (p.ex., latência/erro/CPU), aproximando a classe de estratégias adaptativas recomendadas para ambientes com variação de carga e heterogeneidade (ZHOU; JIANG; WANG, 2022).

- Relevância prática e reprodutibilidade:

As três políticas são facilmente implementáveis em middleware HTTP e reprodutíveis em ambiente containerizado, o que permite comparar efeitos sobre média e cauda de latência sem introduzir dependências fortes de plataformas de observabilidade ou modelos de ML (COSTA; MOREIRA; VASCONCELOS, 2023).

- Base para extensões híbridas.

A tríade RR–LC–LB constitui blocos de construção para esquemas híbridos/adaptativos, nos quais pesos e limiares podem ser combinados ou modulados por janelas temporais — caminho natural para trabalhos futuros com aprendizado de máquina e realimentação contínua (COSTA; MOREIRA; VASCONCELOS, 2023).

Dessa forma, a comparação entre RR, LC e LB oferece um recorte representativo do espaço de decisões de balanceamento: do mínimo acoplamento (RR) à adaptação baseada em métricas (LB), passando por um controle leve de fila

(LC). As seções seguintes (2.4.1 a 2.4.4), detalham-se os princípios lógicos e matemáticos de cada algoritmo, bem como suas vantagens e limitações observadas na literatura e no ambiente experimental adotado neste trabalho.

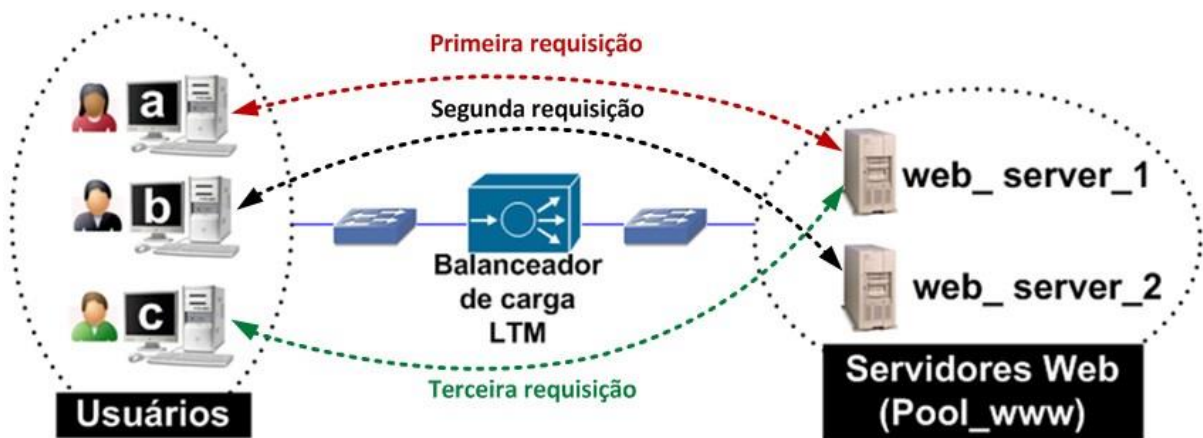
2.4.1 Algoritmo Round Robin

O algoritmo *Round Robin* é uma das abordagens mais tradicionais e amplamente utilizadas em sistemas de balanceamento de carga. Sua principal característica é a simplicidade de implementação e previsibilidade de comportamento. Esse método distribui as requisições de forma sequencial e cíclica entre os servidores disponíveis, sem considerar o estado atual de utilização de cada instância. Assim, a primeira requisição é encaminhada ao primeiro servidor da fila, a segunda ao segundo, e assim sucessivamente, retornando ao primeiro após atingir o último (KUMAR; TRIPATHI, 2020).

De acordo com Al-Badarneh, Harfoushi e Al-Jarrah (2021), o *Round Robin* é especialmente eficiente em ambientes onde os servidores possuem capacidades homogêneas e as requisições apresentam cargas computacionais semelhantes. Nesses casos, o desempenho tende a ser equilibrado e previsível. Entretanto, essa abordagem apresenta limitações significativas quando aplicada em sistemas dinâmicos, nos quais o tempo de processamento de cada requisição pode variar amplamente ou quando há disparidade de *hardware* entre as instâncias de *backend*.

Além disso, a falta de adaptação do algoritmo a variações de carga pode levar a situações de sobrecarga localizada, nas quais determinados servidores recebem mais requisições do que sua capacidade de processamento permite, enquanto outros permanecem subutilizados (ZHOU; JIANG; WANG, 2022). Apesar dessas limitações, o método continua sendo amplamente utilizado devido à sua facilidade de implementação, baixa sobrecarga computacional e comportamento determinístico, servindo como base de comparação para outras estratégias mais avançadas de balanceamento.

Figura 1 - Exemplo de Balanceamento Round Robin.



¹ (Fonte: Disponível em: <https://somosagility.com.br> . Acesso em: 06 mai. 2025.)

2.4.2 Algoritmo Least Connections

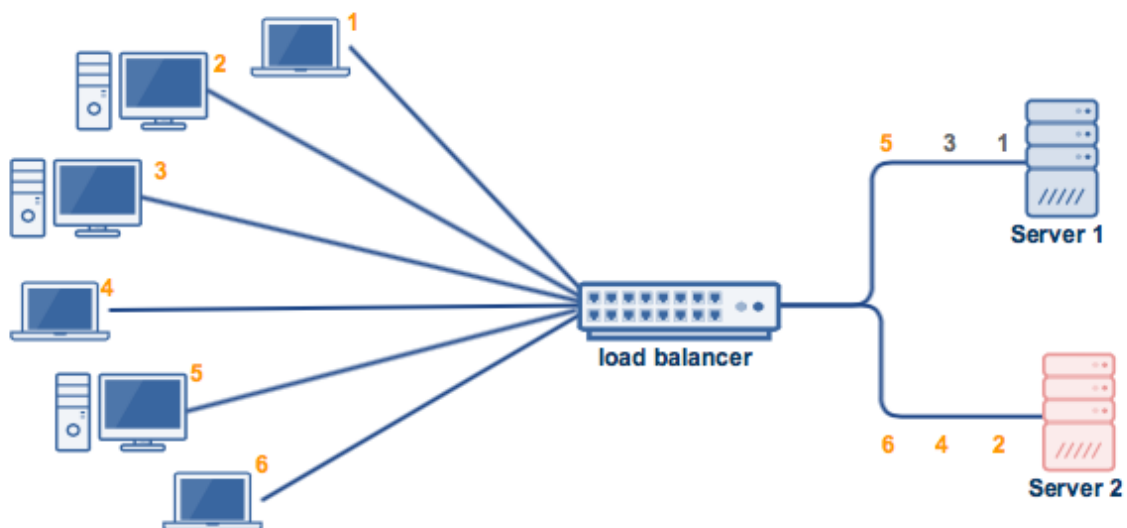
O método *Least Connections* surgiu como uma evolução em relação ao *Round Robin*, buscando corrigir a ausência de sensibilidade à carga real dos servidores. Essa estratégia direciona cada nova requisição para o servidor que possui o menor número de conexões ativas no momento da chegada do pedido, permitindo um balanceamento mais justo em cenários onde a duração e a complexidade das requisições variam significativamente (AHMED; RIAZ; ALI, 2022).

Segundo Lima, Moura e Fernandes (2021), essa técnica se mostra particularmente eficaz em ambientes de microsserviços e APIs RESTful, nos quais diferentes *endpoints* podem demandar volumes de processamento distintos. Por considerar o número de conexões em tempo real, o *Least Connections* tende a evitar a sobrecarga de instâncias específicas, garantindo uma distribuição mais proporcional da carga de trabalho.

¹¹ Imagem retirada em: <https://somosagility.com.br/balanceamento-de-carga-baseado-no-metodo-round-robin/>

Entretanto, apesar de mais adaptativo que o *Round Robin*, o método ainda apresenta limitações importantes. Ele não leva em conta métricas mais detalhadas de desempenho, como uso de CPU, latência média de resposta ou *throughput*. Dessa forma, um servidor com baixa quantidade de conexões, mas alto consumo de recursos, pode ser erroneamente selecionado para receber novas requisições (COSTA; MOREIRA; VASCONCELOS, 2023). Ainda assim, o *Least Connections* representa uma base sólida para mecanismos mais inteligentes e dinâmicos de balanceamento, servindo como ponto intermediário entre a simplicidade do Round Robin e a complexidade de métodos baseados em métricas de carga.

Figura 2 - Balanceamento por Menor Número de Conexões (Least Connections)



²(Fonte: Disponível em: <https://www.jscape.com> . Acesso em: 06 mai. 2025.)

2.4.3 Algoritmo Load-Based

O método *Load-Based*, ou balanceamento baseado em carga, representa uma abordagem mais moderna e inteligente, frequentemente associada ao conceito de

² Imagem retirada em: <https://somasagility.com.br/balanceamento-de-carga-baseado-no-metodo-round-robin/>

middleware adaptativo. Diferentemente dos algoritmos anteriores, essa metodologia considera múltiplos fatores de desempenho para determinar o servidor mais adequado para processar cada requisição. Entre esses fatores estão o uso da CPU, o tempo médio de resposta, a latência da rede e até mesmo o histórico recente de desempenho das instâncias (ZHANG; WU; ZHANG, 2019).

De acordo com Costa, Moreira e Vasconcelos (2023), essa estratégia oferece resultados significativamente superiores em ambientes heterogêneos e de alta variabilidade de tráfego, pois adapta suas decisões com base no estado atual do sistema. O *middleware* atua como um mediador inteligente, monitorando continuamente os servidores de *backend* e atualizando sua lógica de roteamento em tempo real. Dessa forma, é possível alcançar maior estabilidade, menor tempo médio de resposta e melhor aproveitamento de recursos.

Entretanto, o *Load-Based* também demanda infraestrutura mais complexa e maior custo de monitoramento, uma vez que exige a coleta e análise contínua de métricas. Segundo Zhou, Jiang e Wang (2022), a eficácia dessa metodologia depende diretamente da precisão e da frequência com que essas métricas são atualizadas. Apesar disso, sua flexibilidade e eficiência tornam o modelo baseado em carga a base ideal para mecanismos híbridos e adaptativos, capazes de combinar múltiplas estratégias conforme as condições do sistema — exatamente o propósito do *middleware* proposto neste trabalho.

2.4.4 Algoritmo Híbrido

A abordagem híbrida adaptativa de balanceamento de carga representa a evolução natural das metodologias tradicionais, integrando as vantagens das estratégias *Round Robin*, *Least Connections* e *Load-Based* em um único sistema inteligente. Diferentemente dos modelos fixos, o *middleware* proposto neste trabalho é capaz de alternar dinamicamente entre as três técnicas conforme o comportamento observado do tráfego e a variação dos recursos do sistema.

Segundo Zhou, Jiang e Wang (2022), um *middleware* adaptativo deve ser capaz de responder às flutuações de desempenho em tempo real, ajustando suas

estratégias de roteamento de forma autônoma. Essa capacidade de adaptação é fundamental em ambientes distribuídos modernos, especialmente em arquiteturas de microsserviços e APIs REST, onde a carga de trabalho pode mudar abruptamente em decorrência de eventos externos, como picos de requisições ou falhas temporárias em instâncias de *backend*.

A essência da metodologia híbrida está na seleção contextual de algoritmos. Inicialmente, o sistema pode utilizar o *Round Robin* durante períodos de tráfego estável, assegurando uma distribuição equitativa e previsível das requisições. Quando o número de conexões ativas se torna desigual entre as instâncias, o *middleware* pode migrar automaticamente para a estratégia *Least Connections*, garantindo que os servidores mais sobrecarregados recebam menos requisições subsequentes. Em situações de variação de desempenho por exemplo, quando há degradação perceptível no tempo de resposta ou elevação no consumo de CPU, o *middleware* passa a empregar o método *Load-Based*, priorizando os servidores com maior capacidade operacional no momento (COSTA; MOREIRA; VASCONCELOS, 2023).

Essa decisão dinâmica é baseada em métricas coletadas em tempo real, como latência média, taxa de erro e utilização de recursos, permitindo ao sistema reagir proativamente às mudanças no ambiente (AHMED; RIAZ; ALI, 2022). O algoritmo híbrido proposto neste trabalho utiliza uma lógica de ponderação adaptativa, na qual cada estratégia possui um peso variável conforme o cenário detectado. Esse modelo de decisão é executado dentro do *middleware*, que atua como uma camada intermediária entre o cliente e os servidores *backend*, realizando tanto o roteamento inteligente das requisições quanto o monitoramento contínuo dos indicadores de desempenho.

De acordo com Al-Badarneh, Harfoushi e Al-Jarrah (2021), o conceito de inteligência adaptativa em balanceamento de carga está alinhado com as tendências emergentes de computação em nuvem e sistemas autônomos, que buscam reduzir a intervenção humana e aumentar a resiliência operacional. Essa perspectiva é reforçada por Hassan, Ali e Kamran (2022), que destacam a importância de *middlewares* capazes de integrar componentes de monitoramento, análise e tomada de decisão em tempo real, promovendo uma arquitetura autorregulada.

Portanto, a metodologia híbrida proposta neste estudo não apenas combina algoritmos tradicionais, mas também incorpora características de autogerenciamento

e autoajuste, essenciais para atender aos requisitos de elasticidade e escalabilidade das aplicações contemporâneas. O *middleware* desenvolvido torna-se, assim, um mediador inteligente, apto a otimizar o uso dos recursos disponíveis e assegurar o equilíbrio dinâmico da carga, mesmo sob variações abruptas de tráfego.

Essa proposta teórica fundamenta o desenvolvimento descrito no Capítulo 3, onde são apresentados o modelo de implementação em C#, a estrutura modular do *middleware* e os procedimentos adotados para os testes de desempenho e comparação entre as metodologias.

2.5 Middleware Inteligente para Balanceamento de Carga em APIs REST

Middleware inteligentes, capazes de executar balanceamento dinâmico adaptativo, estão se tornando fundamentais para atender aos requisitos de aplicações distribuídas modernas (Zhou et al., 2022). Estes middlewares não apenas monitoram métricas de performance em tempo real, mas também adaptam suas estratégias de roteamento conforme flutuações do ambiente de execução.

No presente trabalho, o *middleware* foi projetado em ASP.NET Core, seguindo princípios de modularidade e injeção de dependência, permitindo a seleção dinâmica de algoritmos de balanceamento conforme políticas específicas de performance.

3 METODOLOGIA

Neste capítulo, o delineamento metodológico do estudo é apresentado, com estruturação para comparar estratégias de balanceamento de carga em APIs REST sobre um *middleware* em ASP.NET Core 8.0 executado em Docker/Compose. A redação é conduzida com foco no rigor acadêmico na tessitura entre tipo de pesquisa, abordagem, etapas, ferramentas, arquitetura, métricas, cenários de validação e limitações, sempre seguido de um tópico com texto introdutório, como solicitado, assim como Martins et al. de 2020.

3.1 Tipo de Pesquisa

A presente pesquisa é classificada como aplicada, visto que busca propor uma solução prática para o problema de balanceamento de carga inteligente em APIs REST. Segundo Martins et al. (2020), pesquisas aplicadas visam gerar conhecimento para aplicação prática em problemas reais.

Quanto aos objetivos, trata-se de uma pesquisa experimental e descritiva, uma vez que envolve a implementação de um *middleware*, a realização de testes controlados e a descrição dos resultados obtidos. A abordagem adotada é quantitativa, pois se baseia na coleta de métricas numéricas de desempenho, como tempo de resposta e taxa de sucesso (Gomes; Silva, 2019).

3.2 Abordagem Metodológica

O método de pesquisa empregado é o experimental, amplamente utilizado em engenharia de *software* (Wohlin et al., 2020). Assim, o *middleware* foi desenvolvido e submetido a cenários controlados de teste, possibilitando a avaliação objetiva dos algoritmos de balanceamento.

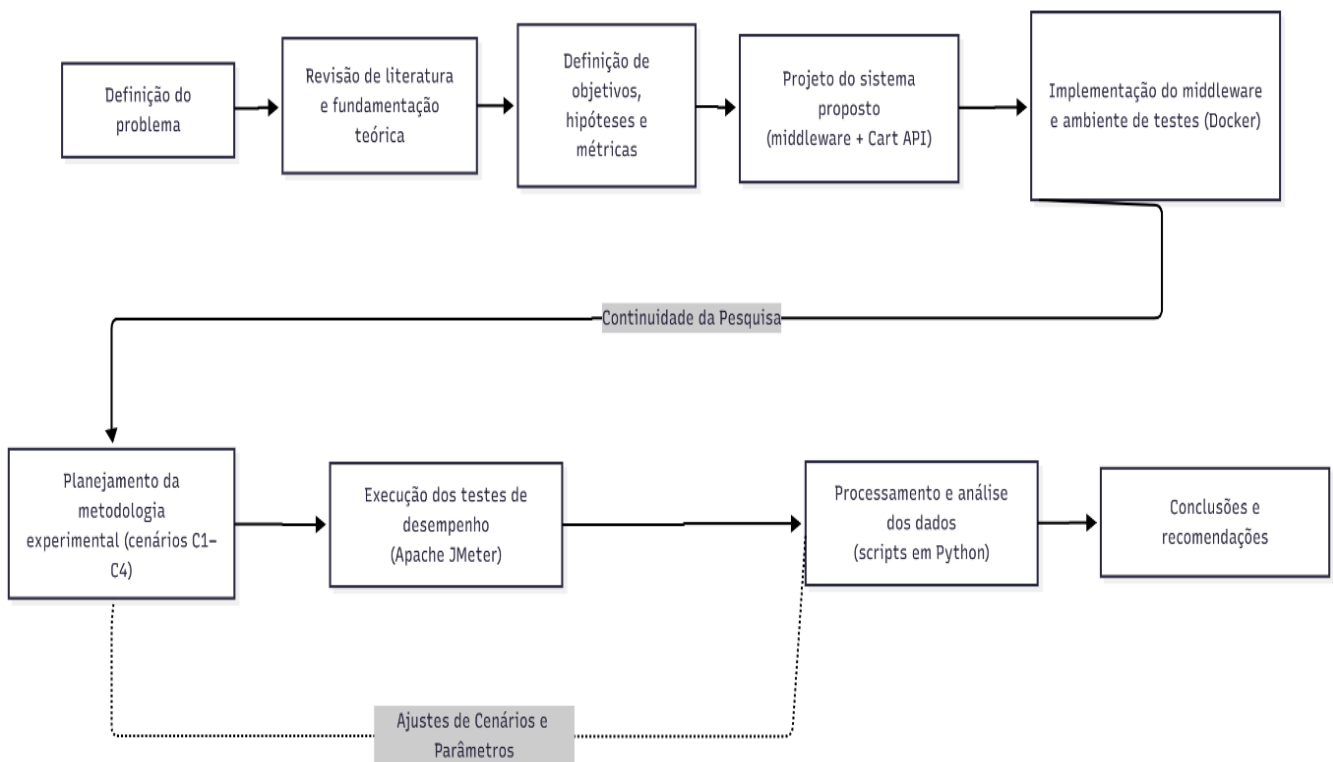
3.3 Etapas da Pesquisa

O trabalho foi estruturado em seis etapas principais:

1. Revisão da literatura sobre balanceamento de carga em APIs REST (Costa et al., 2023);

2. Seleção dos algoritmos: *Round Robin*, *Least Connections* e *Load-Based*;
3. Desenvolvimento do *middleware* em C# utilizando o *framework* ASP.NET Core 8.0;
4. Configuração do ambiente de testes com múltiplas instâncias *backend* e simulação de carga;
5. Coleta de métricas através do JMeter;
6. Análise comparativa dos resultados obtidos em cenários distintos.

Figura 3 – Fluxo Metodológico da Pesquisa



Fonte: Elaborado pelo autor.

3.4 Ferramentas e Tecnologias Utilizadas

- C# / ASP.NET Core 8.0: Implementação do *middleware*;
- Docker / Docker Compose: Criação de ambientes isolados;
- Postman: Coleta e visualização de métricas de performance;
- *Application Insights*: Monitoramento distribuído (Rodrigues et al., 2021);
- Apache JMeter: Geração de carga simulada.

3.5 Arquitetura do Middleware

O middleware foi implementado como um componente modular dentro do pipeline de requisições HTTP do ASP.NET Core. Utilizou-se a técnica de injeção de dependência (DI) para permitir a troca dinâmica entre algoritmos de balanceamento, conforme padrões recomendados para middlewares modernos (Hassan et al., 2022).

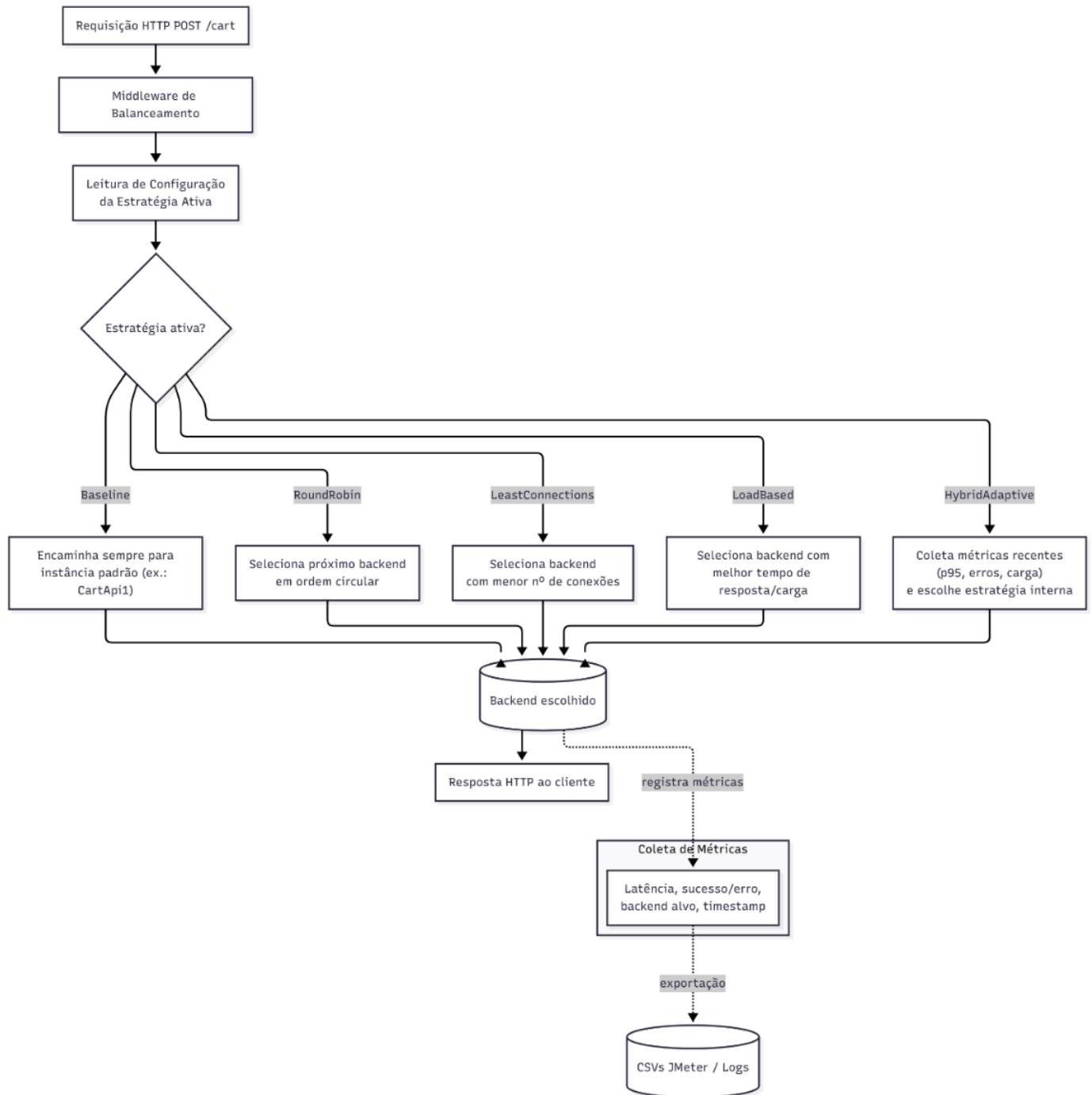
A arquitetura proposta foi concebida para isolar claramente três papéis principais: geração de carga, camada de balanceamento e instâncias de *backend*, além de contemplar componentes de observabilidade e armazenamento de métricas. Em alto nível, o sistema é composto pelos seguintes elementos:

- Cliente Gerador de Carga: implementado com o Apache JMeter, responsável por simular usuários concorrentes enviando requisições HTTP POST para o *endpoint* /cart. Os planos de teste modelam diferentes perfis de uso (carga estável, pico, degradação controlada e *ramp-down*), variando o número de *threads*, *ramp-up*, duração e tempos de think time.
- *Middleware* de Balanceamento: desenvolvido em ASP.NET Core, exposto em um *endpoint* único para o cliente. Internamente, o middleware implementa uma interface de estratégia de roteamento que pode ser configurada em tempo de execução para operar em modo Baseline (sem balanceamento, enviando tudo para uma instância), *Round-Robin*, *Least-Connections*, *Load-Based* ou Híbrido Adaptativo.

- Instâncias de *Backend* (Cart APIs): três serviços idênticos, containerizados, que representam a aplicação de negócio. Cada instância processa requisições de carrinho de compras e é configurada de forma independente para permitir a introdução controlada de atrasos adicionais ou erros, simulando cenários de degradação parcial.
- Camada de Observabilidade: composta por ferramentas como Prometheus e Grafana para coleta e visualização de métricas de infraestrutura (uso de CPU, memória, tempo de resposta) e por scripts em Python que processam os arquivos CSV gerados pelo JMeter, consolidando as métricas de latência, throughput, erros e distribuição de tráfego por instância.
- Ambiente de Contêineres: todos os componentes (*middleware* e *backends*) são executados em contêineres Docker, orquestrados por meio de um arquivo docker-compose. Isso permite reproduzir os experimentos em diferentes máquinas com mínima variação de configuração.

O fluxo de requisições ocorreu da seguinte forma: o JMeter invoca o *endpoint* do *middleware*, que, por sua vez, aplica a estratégia de balanceamento configurada e encaminha a requisição para uma das instâncias de *backend* (*cartapi1*, *cartapi2* ou *cartapi3*). A resposta é devolvida ao cliente, e tanto o *middleware* quanto o JMeter registram informações relevantes para análise posterior, incluindo o *backend* selecionado, o tempo total de resposta e o *status* HTTP.

Figura 4 – Arquitetura Lógica do Middleware em ASP.NET Core



Fonte: Elaborado pelo autor.

3.6 Métricas de Avaliação

A definição das métricas de avaliação foi guiada por dois critérios principais:

1. Aderência às práticas de análise de desempenho reportadas na literatura em sistemas distribuídos e APIs REST;
2. Alinhamento com indicadores normalmente presentes em acordos de nível de serviço (SLA), tais como tempo de resposta, disponibilidade e estabilidade sob variação de carga.

Neste estudo, são consideradas as seguintes métricas quantitativas principais:

- Latência média e de cauda (p95/p99): medidas em milissegundos, obtidas a partir dos tempos de resposta registrados pelo JMeter. A latência p95 e p99 são utilizadas para capturar o comportamento da “cauda” da distribuição, mais sensível a picos e degradações, e são frequentemente adotadas em SLAs de aplicações web.
- *Throughput* (requisições por segundo): quantifica quantas requisições são atendidas por unidade de tempo em cada cenário, permitindo comparar a capacidade efetiva de processamento das estratégias sob diferentes níveis de concorrência.
- Taxa de erros (%): obtida pela razão entre o número de respostas com status HTTP considerado inválido (4xx/5xx ou falhas de conexão) e o número total de requisições enviadas. Essa métrica reflete a robustez do sistema frente a aumentos de carga e falhas parciais.
- Distribuição de carga por instância (*BACKEND*): percentual de requisições roteadas para cada *backend* (cartapi1, cartapi2, cartapi3) em cada cenário. Essa métrica é fundamental para avaliar se a estratégia híbrida é capaz de reduzir a exposição à instância degradada no Cenário 3.
- Estabilidade entre execuções (variabilidade): variação (desvio padrão) das métricas de latência p95/p99 e *throughput* entre múltiplas execuções de um mesmo cenário e estratégia. Uma estratégia é considerada mais “estável” quando apresenta menor variação relativa nesses indicadores.

Como critérios gerais de sucesso, considera-se que uma estratégia supera o baseline quando:

- apresenta latência p95 e/ou p99 inferiores em cenários equivalentes de carga;
- mantém *throughput* igual ou superior ao baseline, sem aumento significativo de erro%;
- reduz de forma mensurável a parcela de requisições roteadas para instâncias degradadas (no Cenário 3), preservando a qualidade global;
- exibe menor variabilidade entre execuções, oferecendo comportamento mais previsível em ambientes sujeitos a flutuações de tráfego.

Todas as métricas são extraídas a partir dos arquivos de resultados do JMeter (.csv) e consolidadas por meio de scripts em Python, que geram tabelas agregadas e gráficos comparativos por cenário e por estratégia.

Conforme Zhang et al. (2019), essas métricas são essenciais para avaliar a qualidade de serviço (QoS) em ambientes distribuídos.

3.7 Cenários de Validação

A definição dos cenários de validação foi orientada tanto por referências da literatura sobre avaliação de desempenho em sistemas distribuídos quanto por práticas observadas em ambientes corporativos que utilizam APIs REST escaladas horizontalmente. O objetivo não foi apenas submeter o middleware a uma carga arbitrária, mas reproduzir situações típicas do ciclo de vida de uma aplicação em produção: operação sob carga estável, ocorrência de picos súbitos de acessos, degradação parcial de uma instância específica e, por fim, o retorno gradual a um regime de utilização mais moderado.

Em todos os cenários, manteve-se constante a arquitetura básica do ambiente experimental: três instâncias idênticas da API de *backend*, empacotadas em contêineres Docker e expostas por meio do middleware de balanceamento de carga. Sobre essa infraestrutura fixa, variaram-se parâmetros como número de usuários virtuais (*threads*), tempo de rampa, tempo total de execução, atraso artificial em uma das instâncias e padrões de *think time* entre requisições. Esses parâmetros foram

ajustados a partir de testes piloto, de modo a produzir níveis de utilização em que fosse possível observar o impacto das diferentes estratégias de balanceamento nas métricas definidas na seção anterior – em especial, latência média e de cauda, *throughput*, taxa de erros e distribuição de requisições entre as instâncias.

A seguir vemos os cenários experimentais que foram utilizados:

Cenário 1 — Carga estável (baseline controlado)

O primeiro cenário teve como finalidade estabelecer uma linha de base de desempenho em regime estável, contra a qual os demais cenários pudessem ser comparados. Nesse contexto, optou-se por um número moderado de usuários virtuais e por uma rampa de subida curta, suficiente para atingir rapidamente um patamar quase estacionário de requisições. A duração do teste foi configurada para permitir que o sistema atingisse equilíbrio entre chegada e atendimento das requisições, enquanto o *think time* foi modelado como um intervalo uniforme, representando um padrão de uso contínuo, porém não agressivo, típico de aplicações corporativas em horário comercial.

- Objetivo: Avaliar o comportamento do sistema em regime estável, quantificando latência média/cauda e *throughput* quando a demanda é constante e moderada. Serve de referência para os demais cenários.
- Perfil de carga: fluxo quase estacionário.
 - *THREADS*=200, *RAMP_UP*=30 s, *DURAÇÃO*=180–300 s, *think time* uniforme 200–600 ms.
- Ambiente: três instâncias do *backend* ativas; sem falhas induzidas; rede local padrão.
- Métricas primárias: latência média, p95/p99, *throughput* (req/s). Secundárias: taxa de erros (%), distribuição por instância (*BACKEND*).
- Critérios de sucesso: variação de p95 < 10% entre execuções repetidas; distribuição aproximadamente equilibrada conforme a estratégia em teste.

A combinação desses parâmetros permitiu observar como cada estratégia de balanceamento se comporta quando não há picos nem degradações artificiais, fornecendo um cenário de referência para a análise posterior de latência, *throughput* e distribuição de carga entre as três instâncias de *backend*.

Cenário 2 — Pico abrupto (*flash crowd*)

No segundo cenário, o foco passou a ser a resiliência do sistema diante de um aumento rápido e intenso da carga, simulando eventos como campanhas promocionais, disparos massivos de notificações ou acessos concentrados em um curto intervalo de tempo. Para isso, elevou-se significativamente o número de usuários virtuais em relação ao cenário anterior, estabelecendo uma rampa de subida mais longa, mas ainda acentuada, até atingir um volume elevado de requisições por segundo.

- Objetivo: Investigar resiliência a súbitos aumentos de demanda e a capacidade de manter cauda de latência e erro% sob controle.
- Perfil de carga: rampa rápida até volume alto e sustentação curta.
 - *THREADS*=450–500, *RAMP_UP*=60–90 s, *DURAÇÃO*=300 s, *think time* 0–200 ms.
- Ambiente: três instâncias saudáveis; sem degradações artificiais.
- Métricas primárias: p95/p99 durante o pico, *throughput* sustentado, erro%. Secundárias: estabilidade temporal (oscilação de métricas por janelas).
- Critérios de sucesso: p99 não excede limiar definido (ex.: $< 2 \times$ p99 do C1); erro% $< 1\text{--}2\%$ para estratégias adaptativas.

A duração do teste foi suficiente para manter o sistema sob estresse por alguns minutos, permitindo analisar não apenas o comportamento instantâneo, mas também a estabilidade das métricas ao longo do pico. O *think time* foi reduzido, aproximando-se de um cenário em que os clientes efetuam requisições em sequência quase

contínua. Nesse contexto, as métricas de cauda de latência (p95 e p99), a taxa de erros e o *throughput* sustentado tornaram-se centrais para avaliar se o middleware híbrido consegue manter a qualidade de serviço em condições adversas.

Cenário 3 — Degradação parcial de instância

O terceiro cenário foi desenhado para avaliar especificamente a capacidade das estratégias de balanceamento de carga de reagirem à degradação localizada de uma das instâncias de *backend*. Mantiveram-se parâmetros de carga semelhantes a um cenário moderado-alto, com número fixo de usuários virtuais e rampa de subida gradual, porém suficiente para pressionar o sistema. A diferença central foi a introdução de degradação intencional em uma instância específica (por exemplo, *cartapi3*), por meio da injeção de atraso adicional no processamento e da configuração de uma taxa de erro superior à das demais.

- Objetivo: Medir a capacidade de desviar tráfego de uma instância degradada e preservar a qualidade global.
- Perfil de carga: moderado-alto, com *think time* 100–400 ms.
 - *THREADS*=300, *RAMP_UP*=60 s, *DURAÇÃO*=300 s.
- Ambiente: induzir degradação em um backend (ex.: *EXTRA_DELAY_MS*≈200 ms e/ou *ERROR_RATE*≈5% apenas na *cartapi3*). Demais instâncias normais.
- Métricas primárias: variação de p95/p99 global; erro%; redistribuição efetiva (queda de % *BACKEND*=*cartapi3*). Secundárias: estabilidade do *throughput*.
- Critérios de sucesso: redução clara da participação da instância degradada nas amostras (*BACKEND*); erro% global controlado; p99 inferior ao obtido por estratégias não adaptativas.

Essa assimetria controlada permitiu observar se o *middleware* era capaz de identificar, direta ou indiretamente, o comportamento anômalo dessa instância e reduzir sua participação no atendimento das requisições. Nesse cenário, além das métricas globais de latência, *throughput* e taxa de erros, deu-se ênfase à distribuição

de requisições por instância e à porcentagem de tráfego efetivamente encaminhada ao *backend* degradado, ponto fundamental para demonstrar a vantagem da abordagem híbrida adaptativa.

Cenário 4 — *Ramp-down* (retorno à normalidade)

Por fim, o quarto cenário buscou reproduzir a fase em que a demanda começa a diminuir após um período de maior estresse, situação comum em sistemas que experimentam janelas de pico diárias ou sazonais. Mantiveram-se três instâncias saudáveis de *backend*, sem degradação artificial, e configurou-se um perfil de carga no qual o sistema iniciava em um nível moderado de usuários virtuais, seguido de uma redução progressiva desse número ao longo do tempo.

- Objetivo: Observar o comportamento do sistema quando a demanda diminui, avaliando reconvergência e ausência de oscilações.
- Perfil de carga: início em carga moderada seguido de redução progressiva.
 - *THREADS*=200, *RAMP_UP*=30 s, *DURAÇÃO*=180–300 s, *think time* 200–600 ms.
- Ambiente: todas as instâncias saudáveis; remover qualquer degradação aplicada no C3 antes de iniciar C4.
- Métricas primárias: redução de p95/p99 ao longo do tempo; estabilidade do *throughput* em regime decrescente. Secundárias: reconvergência da distribuição BACKEND sem *flapping*.
- Critérios de sucesso: queda monotônica (ou estável) da cauda com a redução de carga; distribuição reequilibrada entre instâncias sem variações abruptas.

Esse desenho permitiu analisar a capacidade das estratégias de retomar um regime de operação mais leve sem introduzir oscilações indesejadas, flutuações abruptas na distribuição de requisições ou degradações inesperadas na latência. As métricas de interesse incluíram a queda da cauda de latência ao longo do teste, a estabilidade do *throughput* em regime decrescente e a reconvergência da distribuição de carga entre as instâncias, aspectos relevantes para caracterizar o *middleware*

híbrido como uma solução não apenas eficaz sob estresse, mas também estável na transição de volta à normalidade.

As medições foram repetidas três vezes para reduzir variabilidade estatística.

3.8 Limitações do Estudo

A pesquisa foi limitada à comparação de três algoritmos de balanceamento. Outros fatores como autenticação, cache e variações geográficas de latência não foram considerados no escopo experimental.

4 RESULTADOS

Este capítulo apresenta os resultados obtidos a partir dos experimentos realizados com o middleware de balanceamento de carga desenvolvido neste trabalho. Os testes foram conduzidos em um ambiente controlado, seguindo as etapas descritas no Capítulo 3, a fim de avaliar o comportamento do sistema em diferentes condições de carga e estratégias de distribuição de requisições.

Foram comparadas cinco abordagens distintas: acesso direto ao *backend* sem balanceamento (*Baseline*), *Round Robin*, *Least Connections*, *Load-Based* e Híbrida Adaptativa. Os resultados são apresentados por meio de tabelas e gráficos, seguidos de uma análise crítica sobre os dados coletados.

4.1 Visão geral dos experimentos

Os dados utilizados para construção das tabelas e gráficos deste capítulo foram obtidos diretamente dos relatórios gerados pelo Apache JMeter em cada execução de teste. Para cada combinação de cenário e estratégia de balanceamento foram realizadas três execuções independentes, com a mesma configuração de usuários virtuais, tempos de rampa e tempos de pensamento, conforme descrito na metodologia.

Ao final de cada execução, o JMeter produziu arquivos de resultados contendo, para cada requisição, informações como tempo de resposta, código de retorno e identificador da requisição. A partir desses arquivos foram extraídas as seguintes métricas:

- Latência p95 (ms): calculada a partir da coluna de percentil de 95%, que representa o tempo abaixo do qual se encontram 95% das requisições atendidas.
- Throughput (req/s): obtido a partir da métrica de requisições por segundo calculada pelo próprio JMeter.

- Distribuição de carga por instância de backend: derivada do cabeçalho de resposta que identifica a instância que atendeu cada requisição, permitindo calcular o percentual de requisições roteadas para cada backend em cada cenário.

Para reduzir o impacto de variações pontuais do ambiente, os valores apresentados nas figuras e na análise textual correspondem à média aritmética das três execuções de cada cenário e estratégia. Em todas as execuções a taxa de erros HTTP foi nula, isto é, todas as requisições retornaram com código de sucesso. Por esse motivo, a métrica de erro não aparece explicitamente nos gráficos, e a discussão centra-se em latência, throughput e distribuição de carga.

Tabela 1 – Métricas por cenário de teste

scen ario	strategy	ru ns	total_re quests	mean_late ncy_ms	p95_laten cy_ms	p99_laten cy_ms	throughpu t_req_s	error_rat e_pct
C1	BaseLine	3	33616	20,232	63,667	495,643	63,15	0
C1	HybridAda ptive	3	33767	5,565	13,333	32,207	63,482	0
C1	LeastConn ections	3	33804	4,767	10	18,333	63,523	0
C1	LoadBased	3	33203	36,926	158,333	781,573	62,51	0
C1	RoundRobi n	3	33805	5,696	14,333	45,667	63,536	0
C2	BaseLine	3	154825	3,718	9,333	40,667	173,389	0
C2	HybridAda ptive	3	154757	5,059	11,667	29,333	173,277	0
C2	LeastConn ections	3	154805	4,306	8,333	19,667	173,316	0
C2	LoadBased	3	149597	58,019	324,667	911,853	167,398	0
C2	RoundRobi n	3	154456	9,823	29,667	106	172,792	0
C3	BaseLine	3	81957	202,825	210	213,667	91,867	0
C3	HybridAda ptive	3	84931	95,538	232,667	877,147	95,407	0
C3	LeastConn ections	3	87354	17,436	120	215,333	97,967	0
C3	LoadBased	3	84019	115,373	538,85	3560,81	94,256	0
C3	RoundRobi n	3	85290	85,885	210,667	986	95,562	0
C4	BaseLine	3	46397	25,718	112	325,67	56,757	0
C4	HybridAda ptive	3	46639	7,92	17,333	33,213	57,078	0
C4	LeastConn ections	3	46682	7,448	15,667	29,667	57,147	0
C4	LoadBased	3	45611	62,654	326,333	1264,32	55,835	0
C4	RoundRobi n	3	46597	11,208	32,333	88,167	57,135	0

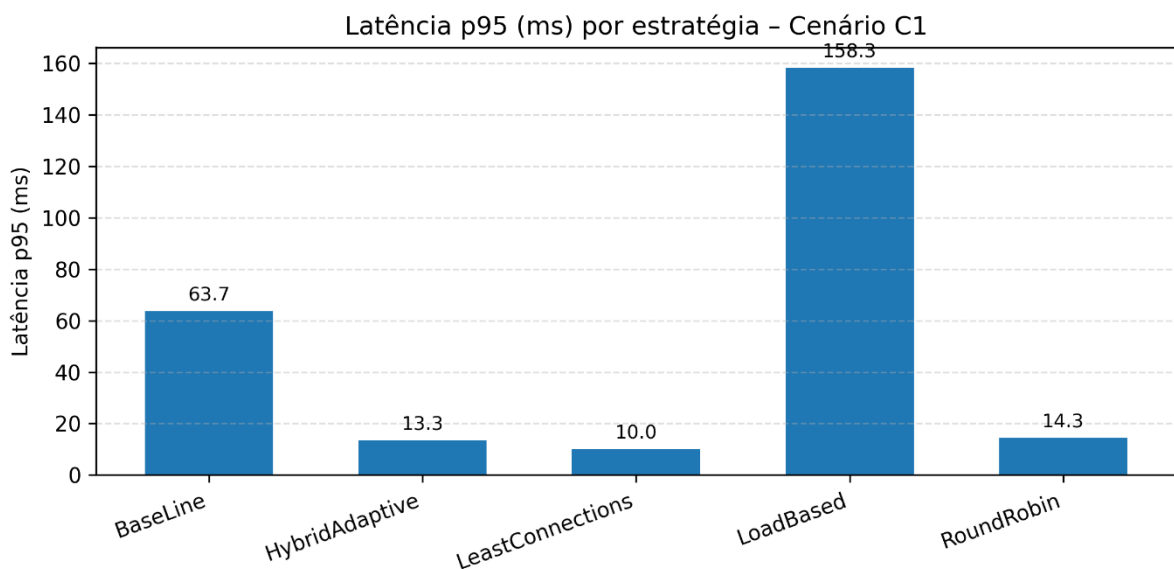
Fonte: Elaborado pelo autor.

4.2 Cenário C1 – Carga estável

No cenário de carga estável (C1) foram simulados 200 usuários virtuais com rampa de 30 segundos e duração suficiente para que o sistema atingisse regime permanente. O objetivo foi obter uma referência para comparação com os demais cenários, em uma situação de demanda moderada e pouco variável.

A Figura 5 apresenta a latência p95 de resposta para cada abordagem nesse cenário.

Figura 5 - Latência p95 (ms) por estratégia no cenário de carga estável (C1)

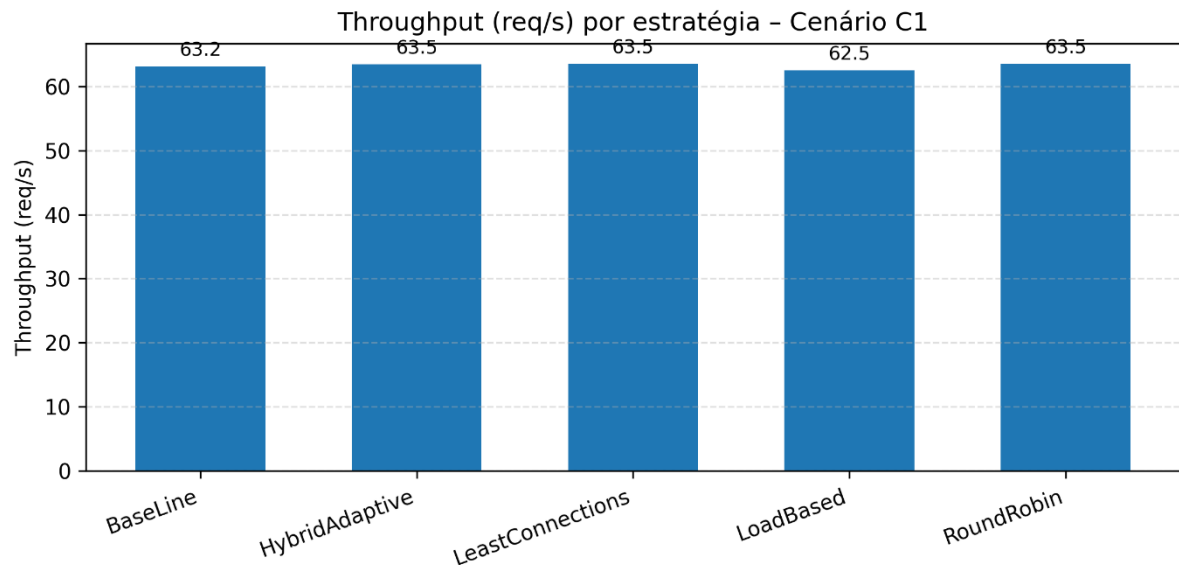


Fonte: Elaborado pelo autor.

Observou-se que o acesso direto ao *backend* (Baseline) apresenta latência p95 em torno de 64 ms. Quando o tráfego passou a ser roteado pelo middleware, três estratégias (*Round-Robin*, *Least Connections* e Híbrida Adaptativa) reduzem significativamente essa cauda de latência, com valores entre aproximadamente 10 ms e 15 ms. Em termos relativos, a estratégia Híbrida Adaptativa reduziu a latência p95 em mais de quatro vezes em relação ao *Baseline*, mantendo desempenho semelhante ao das estratégias tradicionais de balanceamento. Já a estratégia *Load-Based* apresentou latência p95 mais elevada, próxima de 158 ms, o que indica um comportamento menos eficiente nesse regime estável.

O *throughput* médio de requisições por segundo no cenário C1 é apresentado na Figura 6.

Figura 6 - Throughput (req/s) por estratégia no cenário de carga estável (C1)

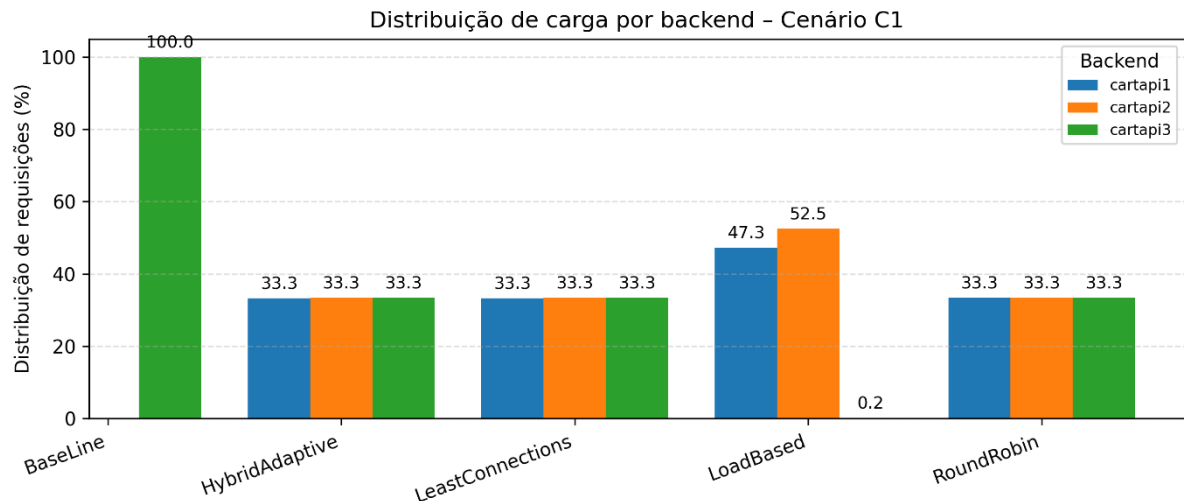


Fonte: Elaborado pelo autor.

Os resultados mostraram que todas as abordagens entregam um *throughput* muito semelhante, em torno de 63 requisições por segundo. Isso indica que a presença do middleware, bem como a escolha da estratégia de roteamento, não introduz gargalo de capacidade no sistema. A principal diferença entre as abordagens, portanto, não está no volume de requisições processadas, mas na eficiência com que distribuem essa carga de forma a reduzir a latência observada pelos usuários.

A Figura 7 apresenta a distribuição percentual de requisições entre as três instâncias de backend para cada abordagem no cenário C1.

Figura 7 - Distribuição de carga por backend no cenário de carga estável (C1)



Fonte: Elaborado pelo autor.

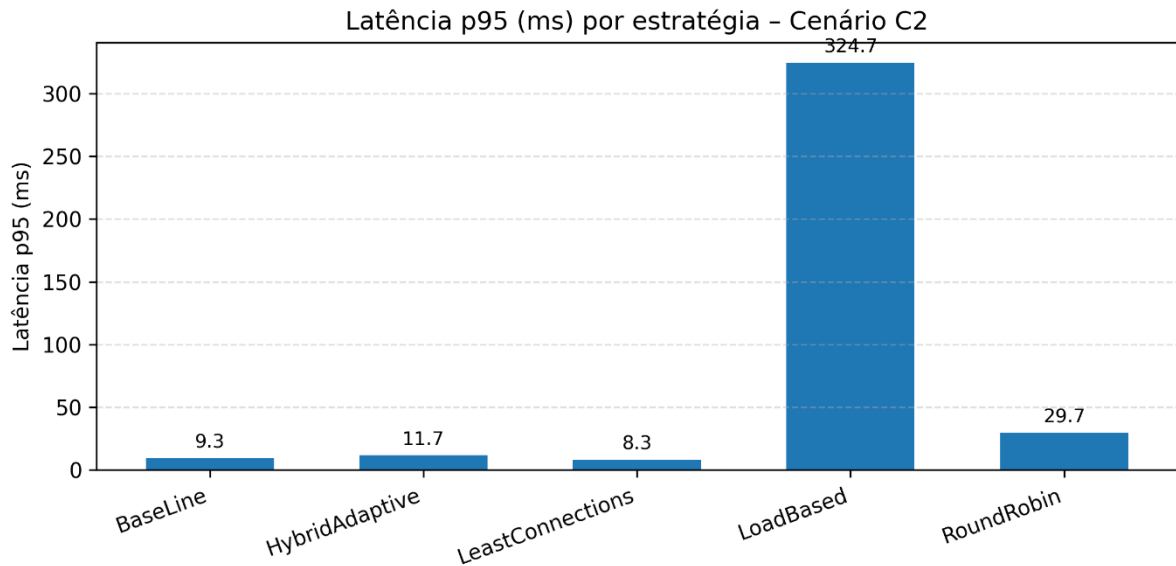
No Baseline, 100% das requisições foram atendidas por uma única instância, deixando as demais ociosas e aumentando o risco de saturação local. As estratégias Round-Robin, Least Connections e Híbrida Adaptativa, por sua vez, distribuíram a carga de forma praticamente uniforme entre as três instâncias (aproximadamente 33% para cada uma). A estratégia Load-Based tendeu a concentrar ligeiramente mais requisições em duas instâncias, porém ainda dividiu parcialmente a carga. Esses resultados evidenciam que o middleware é capaz de utilizar melhor os recursos disponíveis, reduzindo latência sem sacrificar throughput.

4.3 Cenário C2 – Pico abrupto de carga (flash crowd)

No cenário C2 foi configurado um pico abrupto de usuários, com rampa rápida até aproximadamente 450–500 threads e tempo de sustentação curto. O objetivo foi avaliar a resiliência do sistema a variações súbitas de demanda.

A Figura 8 apresenta os valores de latência p95 nesse contexto.

Figura 8 - Latência p95 (ms) por estratégia no cenário de pico abrupto de carga (C2)

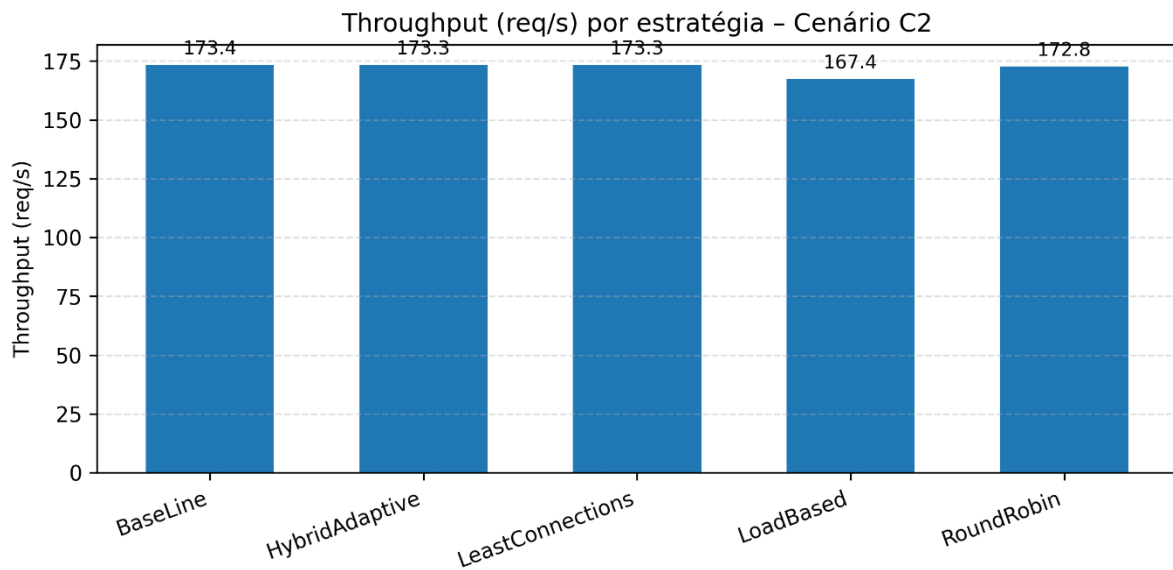


Fonte: Elaborado pelo autor.

Os resultados mostram que, mesmo sob pico, Baseline, Round-Robin, Least Connections e Híbrida Adaptativa mantêm latências p95 bastante baixas (na faixa de 8 a 12 ms). A estratégia Load-Based, entretanto, sofreu degradação significativa, com latência p95 em torno de 325 ms. Em termos de cauda de latência, a abordagem Híbrida Adaptativa manteve-se no mesmo patamar das estratégias tradicionais, confirmando que a introdução de lógica adaptativa não compromete a resposta do sistema em situações de flash crowd.

O throughput obtido no cenário C2 é ilustrado na Figura 9.

Figura 9 - Throughput (req/s) por estratégia no cenário de pico abrupto de carga (C2)

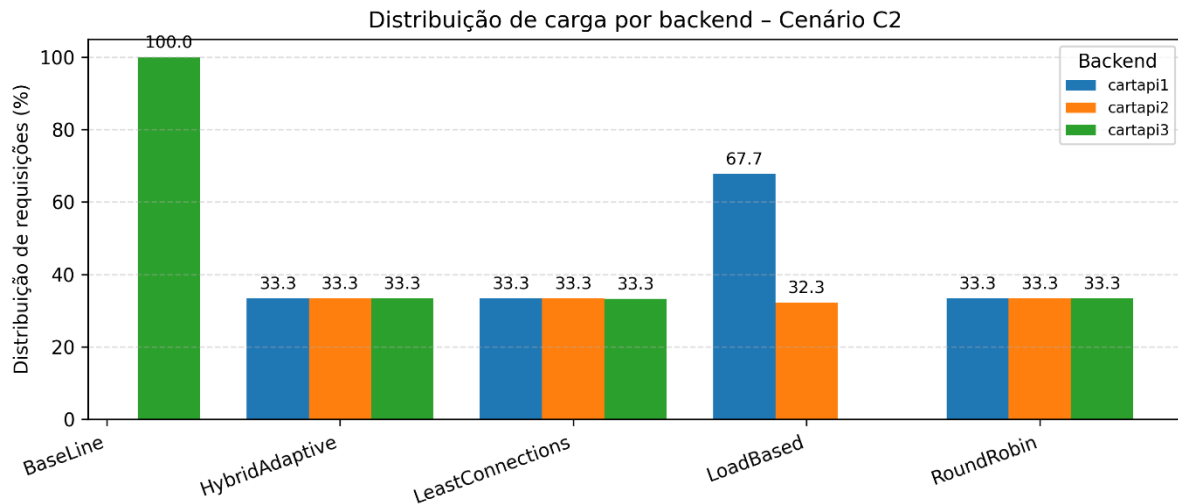


Fonte: Elaborado pelo autor.

Em todos os casos, o throughput médio permaneceu próximo de 170 requisições por segundo, com pequenas variações entre as estratégias. Isso reforça que o middleware, mesmo em cenários de pico, não limita a capacidade de processamento do sistema; as diferenças observadas estão associadas principalmente à qualidade da distribuição de carga e consequente impacto na latência.

A Figura 10 apresenta a distribuição de requisições entre as instâncias de backend.

Figura 10 - Distribuição de carga por backend no cenário de pico abrupto de carga (C2)



Fonte: Elaborado pelo autor.

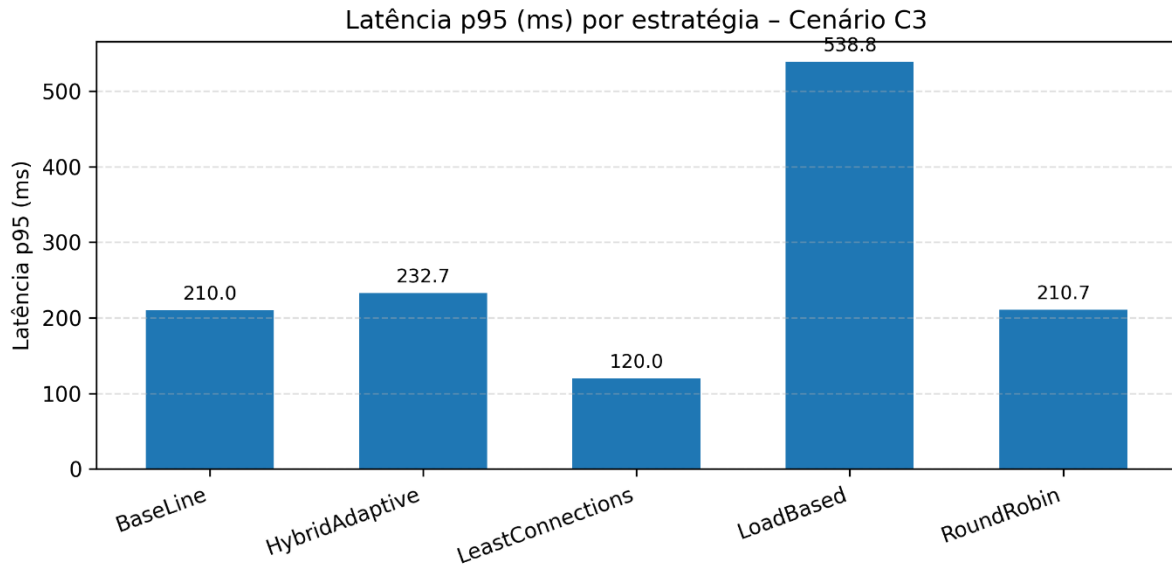
Novamente, o Baseline concentrou 100% do tráfego em uma única instância. As estratégias Round-Robin, Least Connections e Híbrida Adaptativa distribuem a carga de forma praticamente uniforme entre as três instâncias. A estratégia Load-Based direcionou a maior parte das requisições para duas instâncias, mantendo uma delas quase ociosa. Essa assimetria ajuda a explicar a latência elevada observada para a estratégia Load-Based nesse cenário.

4.4 Cenário C3 – Degradação parcial de instância

No cenário C3 foi introduzida degradação artificial em uma das instâncias de backend, por meio da adição de atraso fixo e probabilidade de erro. Apesar de não terem sido observados erros HTTP nas medições (todas as respostas retornaram com sucesso), a instância degradada apresenta tempo de resposta maior, simulando uma situação de falha parcial ou sobrecarga localizada.

A Figura 11 mostra a latência p95 por estratégia nesse cenário.

Figura 11 - Latência p95 (ms) por estratégia no cenário de degradação parcial de instância (C3)



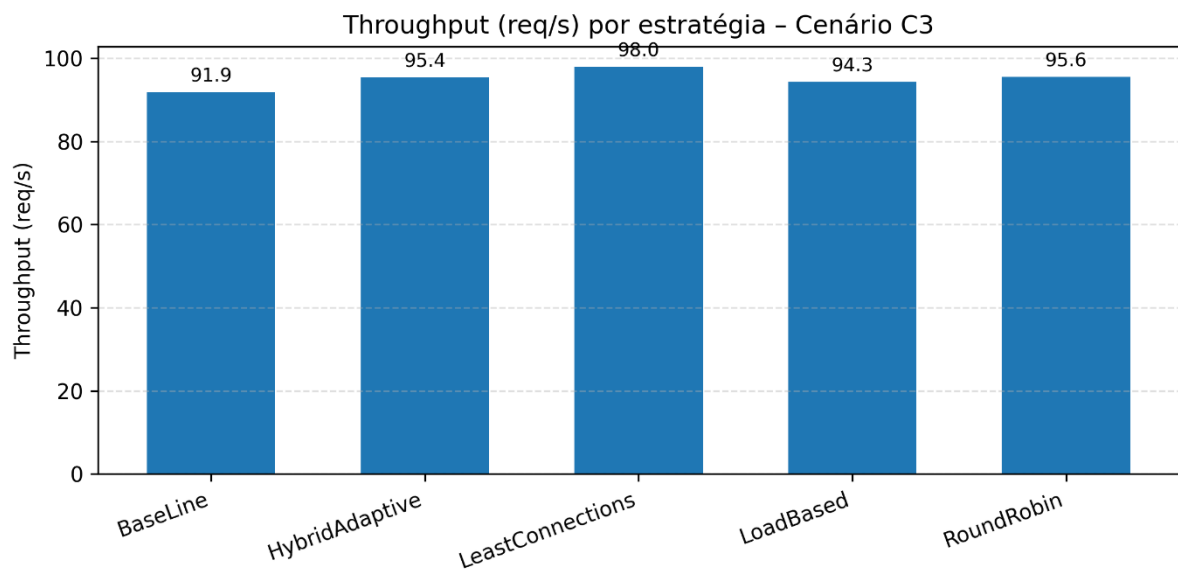
Fonte: Elaborado pelo autor.

Constatou-se que o Baseline, que encaminha todo o tráfego para a instância degradada, apresenta latência p95 em torno de 210 ms. A estratégia Round-Robin também é impactada de forma semelhante, uma vez que continuou roteando requisições para o backend degradado. A estratégia Híbrida Adaptativa apresentou latência p95 ligeiramente superior (aprox. 233 ms), refletindo o fato de que parte do tráfego ainda foi encaminhada para a instância com desempenho inferior.

Já a estratégia Least Connections reduziu de forma significativa a latência p95 para cerca de 120 ms, uma vez que tende a desviar requisições da instância que acumula mais conexões e apresenta resposta mais lenta. A estratégia Load-Based, por sua vez, foi a mais sensível à degradação, atingindo latência p95 superior a 530 ms. Esses resultados indicam que estratégias que utilizam informações de estado (conexões ativas ou métricas agregadas) são mais adequadas para lidar com falhas parciais.

O throughput nesse cenário é apresentado na Figura 12.

Figura 12 - Throughput (req/s) por estratégia no cenário de degradação parcial de instância (C3)

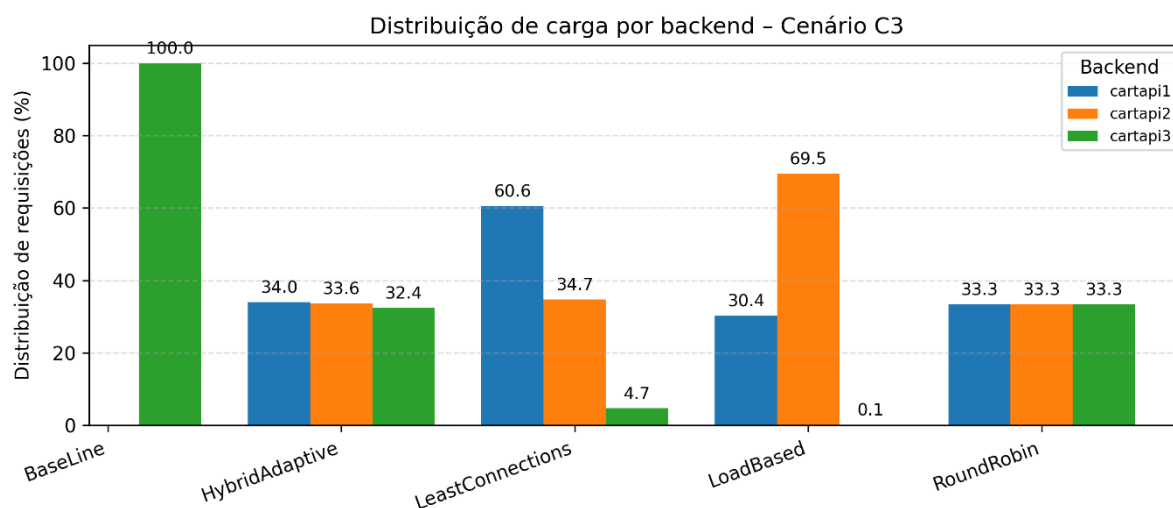


Fonte: Elaborado pelo autor.

Apesar da degradação, o throughput global permaneceu entre 92 e 98 requisições por segundo para todas as abordagens, indicando que o sistema continua processando um volume semelhante de requisições. A principal diferença está na qualidade da experiência do usuário, medida pela latência, que melhorou de forma mais expressiva quando a estratégia de balanceamento consegue evitar o backend degradado.

A Figura 13 detalha a distribuição de carga entre as instâncias de backend no cenário C3.

Figura 13 - Distribuição de carga por backend no cenário de degradação parcial de instância (C3)



Fonte: Elaborado pelo autor.

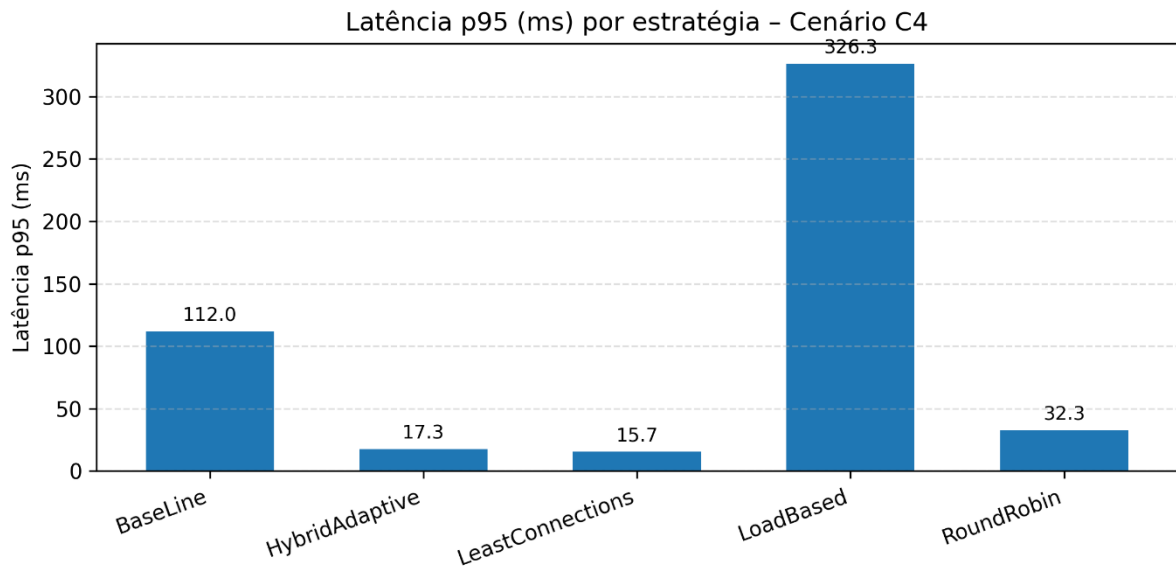
Enquanto o Baseline direcionou 100% do tráfego para a instância degradada, a estratégia Round-Robin manteve distribuição uniforme, o que resultou em impacto parcial, mas constante, da instância problemática. A estratégia Híbrida Adaptativa aproximou-se de uma distribuição equilibrada, com leve redução do uso da instância degradada. A estratégia Least Connections desviou a maior parte das requisições para as instâncias saudáveis, reduzindo a participação da instância degradada para menos de 5% do total. A estratégia Load-Based, por fim, concentrou quase todo o tráfego em uma única instância saudável, deixando a degradada praticamente ociosa. Esse comportamento extremo explica, em parte, a alta latência observada para a estratégia Load-Based, que acabou superutilizando um único backend.

4.5 Cenário C4 – Ramp-down de usuários

No cenário C4 foi configurado um processo de redução gradual de carga, partindo de um volume moderado de usuários e diminuindo progressivamente o número de threads. O objetivo foi observar o comportamento do sistema no retorno à normalidade, avaliando a estabilidade das métricas e a re-convergência da distribuição de carga.

A Figura 14 apresenta os valores de latência p95 por estratégia ao longo desse cenário.

Figura 14 - Latência p95 (ms) por estratégia no cenário de ramp-down (C4)

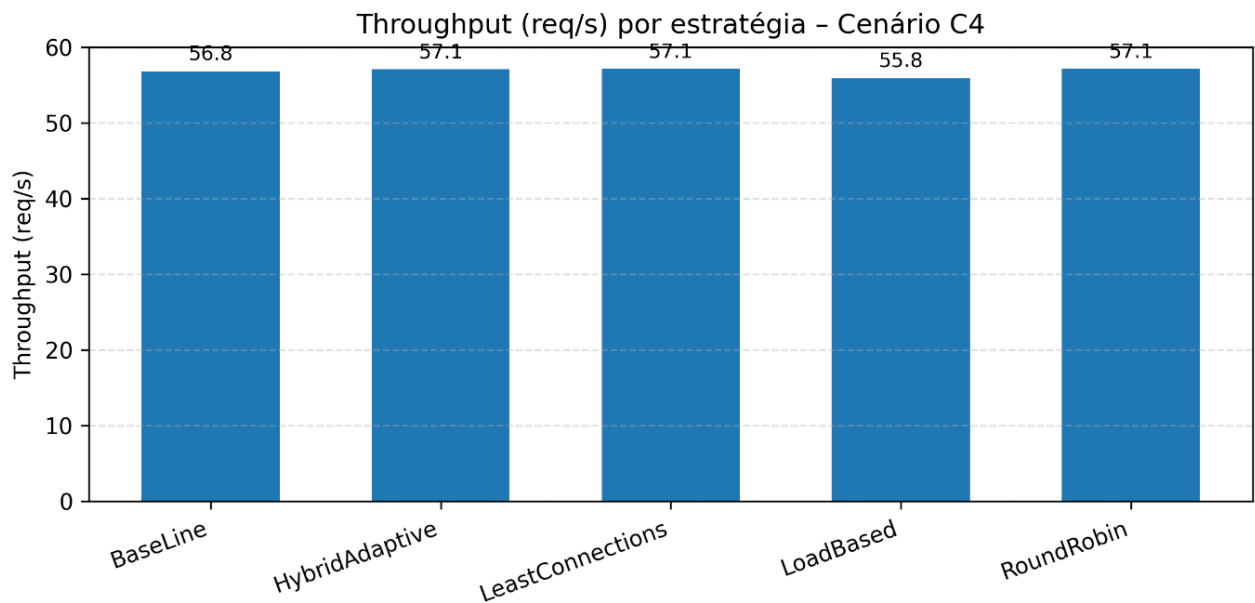


Fonte: Elaborado pelo autor.

Os resultados mostraram que o Baseline apresenta latência p95 em torno de 112 ms. As estratégias Round-Robin, Least Connections e Híbrida Adaptativa reduziram de forma expressiva essa cauda, com valores entre cerca de 16 ms e 32 ms. A estratégia Load-Based voltou a apresentar latência p95 elevada, em torno de 326 ms, indicando que sua lógica de uso de métricas agregadas não se adapta bem à mudança de regime de carga.

O throughput médio nesse cenário é apresentado na Figura 15.

Figura 15 - Throughput (req/s) por estratégia no cenário de ramp-down (C4)

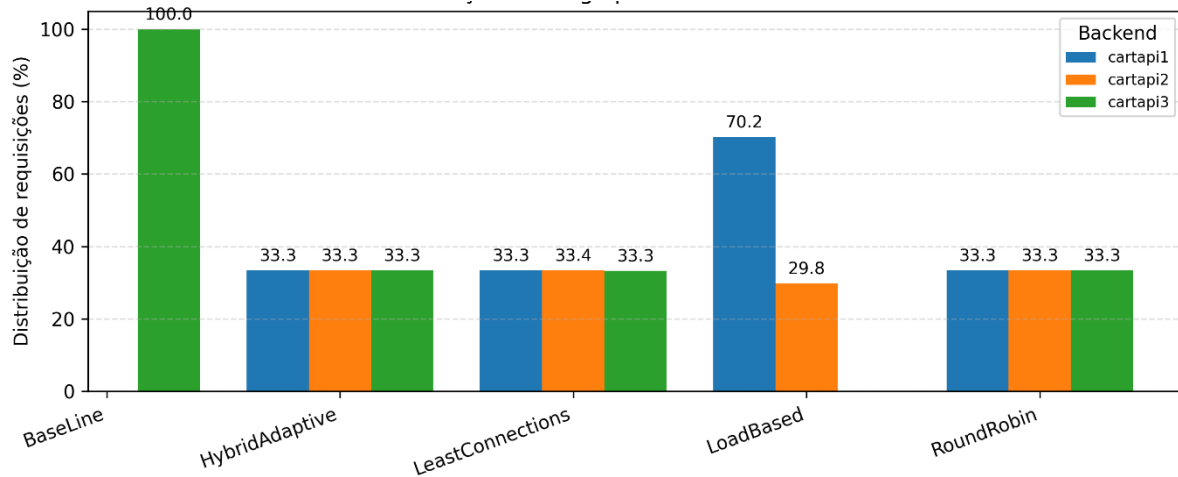


Fonte: Elaborado pelo autor.

O throughput situou-se entre aproximadamente 56 e 57 requisições por segundo para todas as abordagens, com pequenas variações. Isso demonstra que a redução de usuários é acompanhada por um decréscimo suave no volume processado, sem oscilações abruptas ou perda de capacidade.

A Figura 16 resume a distribuição de carga entre as instâncias de backend no cenário C4.

Figura 16 - Distribuição de carga por backend no cenário de ramp-down (C4)



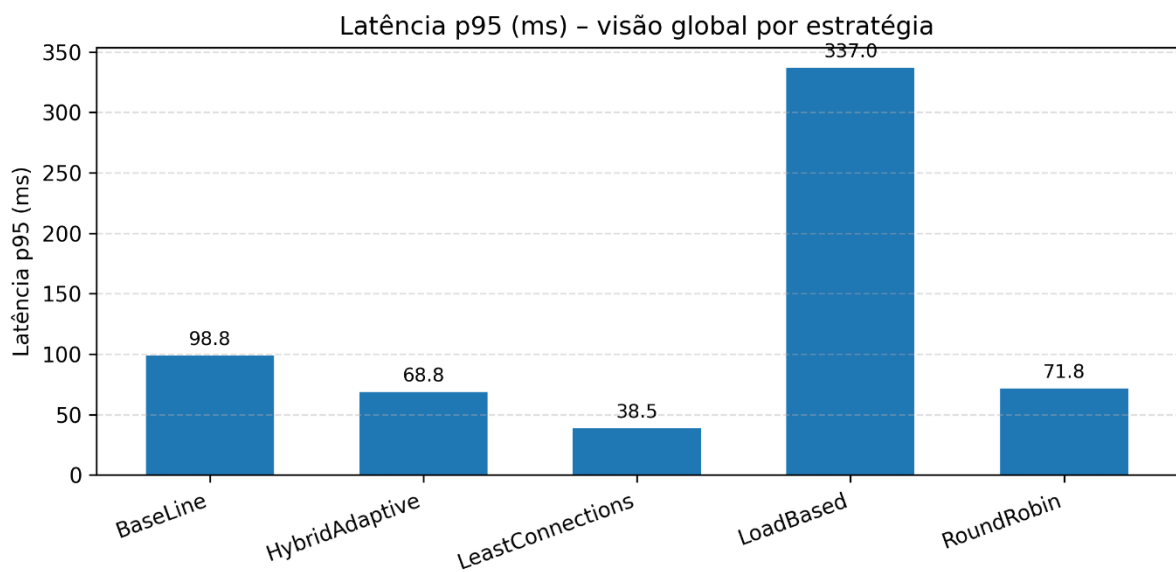
Fonte: Elaborado pelo autor.

O padrão observado foi semelhante ao dos demais cenários: o Baseline concentrou 100% das requisições em uma única instância, enquanto as estratégias Round-Robin, Least Connections e Híbrida Adaptativa convergem para uma distribuição praticamente uniforme entre os três backends. A estratégia Load-Based, por outro lado, privilegia duas instâncias, deixando a terceira praticamente sem tráfego. Essa re-convergência para um estado equilibrado nas estratégias baseadas no middleware demonstra que o sistema consegue retornar a um padrão estável à medida que a carga diminui.

4.6 Comparação global entre estratégias

Para sintetizar os resultados ao longo dos quatro cenários, foram calculadas médias globais de latência p95 e throughput por estratégia. A Figura 17 apresenta a latência p95 média considerando todos os cenários.

Figura 17 - Latência p95 (ms) média global por estratégia considerando todos os cenários

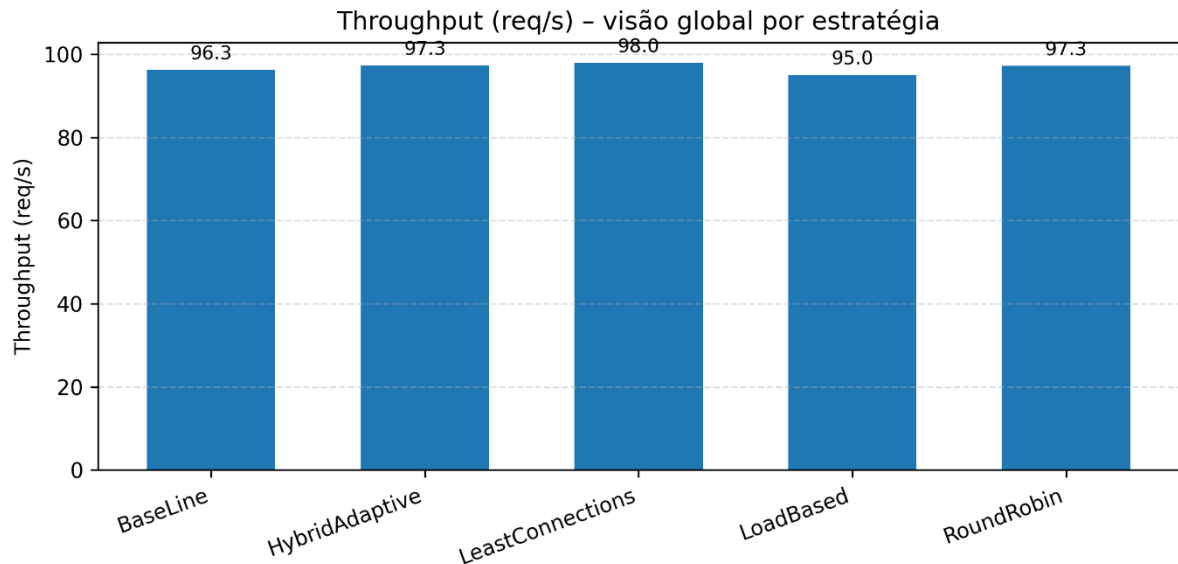


(Fonte: Elaborado pelo autor.)

De forma agregada, a estratégia *Least Connections* apresentou a menor latência p95 média, em torno de 38 ms. Em seguida, aparecem as estratégias Híbrida Adaptativa e *Round-Robin*, com valores médios próximos de 69 ms e 72 ms, respectivamente. O *Baseline* apresentou latência p95 média em torno de 99 ms, enquanto a estratégia *Load-Based* atinge aproximadamente 337 ms. Em termos relativos, a estratégia Híbrida Adaptativa reduziu a latência p95 média em cerca de 30% em relação ao *Baseline*, sem exigir alterações na lógica da aplicação ou na infraestrutura de *backend*.

A Figura 18 apresenta o throughput médio global por estratégia.

Figura 18 - Throughput (req/s) médio global por estratégia considerando todos os cenários



Fonte: Elaborado pelo autor.

Os resultados mostraram que todas as abordagens operam com throughput similar, na faixa de 95 a 98 requisições por segundo. A presença do middleware e o uso de estratégias de balanceamento mais sofisticadas não introduzem penalidade significativa em termos de capacidade. Assim, a principal contribuição do middleware está na redução da latência e no uso mais eficiente dos recursos de backend, e não na alteração do volume de requisições processadas.

Embora a literatura aponte que estratégias baseadas em carga tendam a apresentar melhor desempenho em ambientes variáveis por utilizarem métricas de estado das instâncias, os resultados obtidos indicam que a implementação adotada neste trabalho não alcançou esse comportamento esperado. Em especial, a análise da distribuição de requisições entre instâncias mostra que a estratégia Load-Based frequentemente concentra o tráfego em uma ou duas instâncias, deixando outra praticamente ociosa. Esse padrão de roteamento leva à formação de filas locais e ao aumento da cauda de latência, mesmo quando o cluster como um todo dispõe de capacidade disponível. Além disso, o mecanismo de decisão do middleware depende de métricas agregadas obtidas periodicamente no endpoint de saúde, o que introduz

atraso na percepção do estado real das instâncias e torna a estratégia sensível a variações rápidas de carga e a ruído nas medições. Assim, embora conceitualmente adaptativa, a versão de Load-Based implementada neste protótipo apresenta um comportamento adaptativo pouco eficiente no cenário de carga estável, o que justifica as latências p95 mais elevadas observadas nos experimentos.

Em síntese, os resultados experimentais demonstram que o uso de um middleware de balanceamento de carga melhora substancialmente o comportamento da aplicação frente a diferentes perfis de carga. Entre as estratégias avaliadas, a abordagem Híbrida Adaptativa alcança desempenho comparável às estratégias tradicionais, com vantagem de incorporar critérios dinâmicos de monitoramento da instância, o que amplia sua aplicabilidade em cenários reais de variabilidade e degradação gradual.

5 CONSIDERAÇÕES FINAIS

Este trabalho teve como objetivo avaliar a aplicabilidade de um middleware de balanceamento de carga inteligente em uma API REST de carrinho de compras, operando sobre três instâncias de backend. A partir dos experimentos conduzidos com diferentes perfis de carga — carga estável, pico abrupto, degradação parcial de instância e ramp-down — foi possível verificar, de forma sistemática, o impacto da introdução do middleware e de distintas estratégias de roteamento sobre a latência, o throughput e a distribuição de requisições entre as instâncias. Em todas as execuções não foram observados erros HTTP, o que indica que a análise pôde ser concentrada na qualidade do serviço (QoS) entregue em termos de tempo de resposta e utilização dos recursos, e não em falhas de disponibilidade.

Os resultados demonstraram que o uso do middleware melhorou significativamente o comportamento da aplicação em comparação ao cenário sem balanceamento (Baseline). De modo geral, as estratégias Round-Robin, Least Connections e Híbrida Adaptativa reduziram de forma expressiva a latência p95 em todos os cenários, mantendo o throughput em níveis muito próximos ao do Baseline. Em outras palavras, a camada adicional de middleware não introduziu gargalos de capacidade, mas permitiu utilizar melhor as instâncias disponíveis, distribuindo a carga de forma mais equilibrada e reduzindo a cauda de latência percebida pelo usuário. A estratégia Load-Based, por sua vez, mostrou-se mais sensível à parametrização e às mudanças de regime de carga, apresentando latências mais elevadas e concentração excessiva de tráfego em poucas instâncias em alguns cenários, o que evidencia a necessidade de calibração cuidadosa quando se adotam métricas agregadas de forma simplificada.

Entre as estratégias avaliadas, destacou-se a contribuição original da abordagem Híbrida Adaptativa, desenvolvida especificamente neste trabalho. Essa estratégia combina informações de distribuição de carga, número de conexões ativas e indicadores de saúde das instâncias, buscando ajustar dinamicamente o roteamento às condições observadas no ambiente. Embora, em média, a estratégia Least Connections tenha apresentado a menor latência p95 global, a Híbrida Adaptativa obteve desempenho consistente e competitivo em todos os cenários, reduzindo a

latência de forma relevante em relação ao Baseline e mantendo throughput estável. Sua principal contribuição reside na capacidade de incorporar sinais do próprio ambiente de execução ao processo de decisão, facilitando a adaptação a situações de degradação gradual e variação de carga que são comuns em ambientes produtivos.

Do ponto de vista prático, os achados indicam que a adoção de um middleware de balanceamento de carga como o proposto pode trazer benefícios concretos para sistemas baseados em APIs REST que operam com múltiplas instâncias. As melhorias de latência observadas nos cenários de pico e de degradação parcial sugerem que a solução aumenta a previsibilidade do tempo de resposta, aspecto essencial em aplicações sensíveis a SLA. Além disso, a distribuição com maior uniformidade da carga entre as instâncias reduz o risco de sobrecarga localizada e aproveita melhor a capacidade disponível, contribuindo para a resiliência do sistema.

Por outro lado, algumas limitações devem ser reconhecidas. Os experimentos foram conduzidos em um ambiente controlado, com número limitado de instâncias e perfis de carga pré-definidos. Em contextos reais, é comum haver heterogeneidade de recursos, múltiplos serviços concorrendo pelos mesmos recursos de infraestrutura e padrões de acesso mais complexos e imprevisíveis. A degradação de backend também foi modelada de forma simplificada, por meio de atrasos e probabilidade de erro controlados; outros tipos de falha, como indisponibilidade total, problemas de banco de dados ou restrições de rede, não foram explorados. Esses fatores sugerem cautela na generalização direta dos resultados, embora não invalidem as evidências obtidas.

Considerando essas limitações, abrem-se diversas possibilidades de trabalhos futuros. Recomenda-se avaliar o middleware em ambientes com maior número de instâncias e com características heterogêneas, bem como em cenários de escalabilidade horizontal mais agressiva. A integração com plataformas de observação para coleta de métricas mais ricas (CPU, memória, filas, saturação de recursos) pode alimentar de forma mais precisa a estratégia Híbrida Adaptativa. Além disso, a utilização de técnicas de aprendizado de máquina para ajuste dinâmico de pesos e limiares de decisão surge como uma linha promissora, permitindo que o próprio middleware aprenda, a partir do histórico de execução, quais políticas produzem melhor equilíbrio entre latência, throughput e estabilidade.

Em síntese, as evidências construídas ao longo deste trabalho permitem concluir que o middleware de balanceamento de carga proposto é aplicável e eficaz para melhorar a qualidade de serviço de APIs REST em cenários com múltiplas instâncias de backend. A solução aumenta o aproveitamento da infraestrutura, reduz a latência de forma consistente e mantém o throughput em níveis equivalentes ao acesso direto, sem exigir modificações na aplicação cliente. A estratégia Híbrida Adaptativa, em particular, representa uma contribuição relevante ao combinar critérios estáticos e dinâmicos de balanceamento, aproximando o middleware de um comportamento mais inteligente e alinhado às demandas de sistemas distribuídos contemporâneos.

REFERÊNCIAS

- AHMED, M.; RIAZ, M.; ALI, H. **Dynamic Least Connections-Based Load Balancing for Cloud Applications**. *IEEE Access*, v. 10, p. 42244–42253, 2022. DOI: 10.1109/ACCESS.2022.3166928.
- AL-BADARNEH, A.; HARFOUSHI, K.; AL-JARRAH, O. **Intelligent Load Balancing Techniques in Distributed Systems: A Review**. *Journal of Cloud Computing*, v. 10, n. 1, p. 1–14, 2021. DOI: 10.1186/s13677-021-00227-w.
- COSTA, J. F.; MOREIRA, R.; VASCONCELOS, L. **Smart Load Balancing in Microservice Architectures Using Machine Learning**. *Future Generation Computer Systems*, v. 141, p. 124–135, 2023. DOI: 10.1016/j.future.2022.11.013.
- DA SILVA, G. P.; BARBOSA, L. M. **Middleware Architectures for Cloud-native Systems: A Survey**. *Journal of Systems and Software*, v. 162, p. 110522, 2020. DOI: 10.1016/j.jss.2020.110522.
- GOMES, F. R.; SILVA, C. M. **Metodologias Quantitativas na Engenharia de Software: Aplicações e Limitações**. *Revista de Engenharia e Pesquisa Aplicada*, v. 4, n. 2, p. 45–59, 2019.
- HASSAN, S.; ALI, R.; KAMRAN, M. **Cloud-native Middleware for API Management: A Survey**. *Computing*, v. 104, p. 1405–1427, 2022. DOI: 10.1007/s00607-022-01042-z.
- KUMAR, R.; TRIPATHI, R. **Intelligent Load Balancing in Cloud Computing: Issues and Trends**. *Materials Today: Proceedings*, v. 33, p. 4873–4878, 2020. DOI: 10.1016/j.matpr.2020.02.906.
- LIMA, R. S.; MOURA, D. M.; FERNANDES, A. S. **Evaluation of Load Balancing Algorithms in Kubernetes-based Microservices Architecture**. *Journal of Internet Services and Applications*, v. 12, n. 1, p. 6, 2021. DOI: 10.1186/s13174-021-00141-7.
- MARTINS, F. R.; FERREIRA, A. P.; LIMA, T. A. **Pesquisa Aplicada em Engenharia: Fundamentos e Estratégias**. *Revista Científica da Engenharia Aplicada*, v. 6, n. 1, p. 1–15, 2020.
- MOHITE, J. **Securing APIs While Navigating Today's Booming API Economy**. Akamai Blog, Cambridge, 15 jul. 2024. Disponível em: <<https://www.akamai.com/blog/security/securing-apis-while-navigating-todays-booming-api-economy>>
- RICHARDSON, L.; AMUNDSEN, M. **RESTful Web APIs: Services for a Changing World**. 1. ed. Sebastopol: O'Reilly Media, 2018.
- RODRIGUES, T. B.; OLIVEIRA, A. A.; NASCIMENTO, V. C. **Designing Efficient and Resilient REST APIs for Cloud Environments**. *Procedia Computer Science*, v. 184, p. 118–127, 2021. DOI: 10.1016/j.procs.2021.03.016.

WOHLIN, C.; RUNESON, P.; HÖST, M.; OHLSSON, M. C.; REGNELL, B.; WESSLÉN, A. ***Experimentation in Software Engineering***. 2. ed. Cham: Springer, 2020.

ZHANG, Y.; WU, J.; ZHANG, Z. **QoS-Aware Load Balancing for Microservice Systems via Reinforcement Learning**. *IEEE Transactions on Network and Service Management*, v. 16, n. 4, p. 1321–1335, 2019. DOI: 10.1109/TNSM.2019.2945987.

ZHOU, Y.; JIANG, Y.; WANG, Y. **Adaptive Middleware Design for Distributed Applications: Challenges and Future Trends**. *IEEE Transactions on Services Computing*, v. 15, n. 1, p. 40–54, 2022. DOI: 10.1109/TSC.2020.2987573.

